

SPINN: Synergistic Progressive Inference of Neural Networks over Device and Cloud

Stefanos Laskaridis^{†*}, Stylianos I. Venieris^{†*},
 Mario Almeida^{†*}, Ilias Leontiadis^{†*}, Nicholas D. Lane^{†,‡}
[†]Samsung AI Center, Cambridge [‡]University of Cambridge
 * Indicates equal contribution.
 {stefanos.l,s.venieris,mario.a,i.leontiadis,nic.lane}@samsung.com

ABSTRACT

Despite the soaring use of convolutional neural networks (CNNs) in mobile applications, uniformly sustaining high-performance inference on mobile has been elusive due to the excessive computational demands of modern CNNs and the increasing diversity of deployed devices. A popular alternative comprises offloading CNN processing to powerful cloud-based servers. Nevertheless, by relying on the cloud to produce outputs, emerging mission-critical and high-mobility applications, such as drone obstacle avoidance or interactive applications, can suffer from the dynamic connectivity conditions and the uncertain availability of the cloud. In this paper, we propose SPINN, a distributed inference system that employs synergistic device-cloud computation together with a progressive inference method to deliver fast and robust CNN inference across diverse settings. The proposed system introduces a novel scheduler that co-optimises the early-exit policy and the CNN splitting at run time, in order to adapt to dynamic conditions and meet user-defined service-level requirements. Quantitative evaluation illustrates that SPINN outperforms its state-of-the-art collaborative inference counterparts by up to 2× in achieved throughput under varying network conditions, reduces the server cost by up to 6.8× and improves accuracy by 20.7% under latency constraints, while providing robust operation under uncertain connectivity conditions and significant energy savings compared to cloud-centric execution.

CCS CONCEPTS

• **Computing methodologies** → *Distributed computing methodologies*; • **Human-centered computing** → *Ubiquitous and mobile computing*.

KEYWORDS

Deep neural networks, distributed systems, mobile computing

1 INTRODUCTION

With the spectrum of CNN-driven applications expanding rapidly, their deployment across mobile platforms poses significant challenges. Modern CNNs [20, 68] have excessive computational demands that hinder their wide adoption in resource-constrained mobile devices. Furthermore, emerging user-facing [80] and mission-critical [37, 42, 66] CNN applications require low-latency processing to ensure high quality of experience (QoE) [4] and safety [11].

Given the recent trend of integrating powerful System-on-Chips (SoCs) in consumer devices [2, 25, 78], direct on-device CNN execution is becoming possible (Figure 1 - top left). Nevertheless,

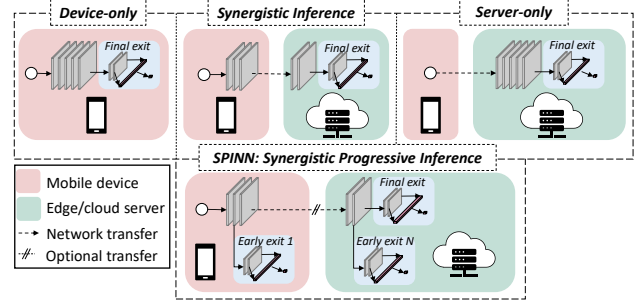


Figure 1: Existing methods vs. SPINN.

while flagship devices can support the performance requirements of CNN workloads, the current landscape is still very diverse, including previous-gen and low-end models [80]. In this context, the less powerful low-tier devices struggle to consistently meet the application-level performance needs [2].

As an alternative, service providers typically employ cloud-centric solutions (Figure 1 - top right). With this setup, inputs collected by mobile devices are transmitted to a remote server to perform CNN inference using powerful accelerators [3, 6, 12, 19, 31, 32]. However, this extra computation capability comes at a price. First, cloud execution is highly dependent on the dynamic network conditions, with performance dropping radically when the communication channel is degraded. Second, hosting resources capable of accelerating machine learning tasks comes at a significant cost [40]. Moreover, while public cloud providers offer elastic cost scaling, there are also privacy and security concerns [64].

To address these limitations, a recent line of work [22, 34, 46] has proposed the collaboration between device and cloud for CNN inference (Figure 1 - top center). Such schemes typically treat the CNN as a computation graph and partition it between device and cloud. At run time, the client executes the first part of the model and transmits the intermediate results to a remote server. The server continues the model execution and returns the final result back to the device. Overall, this approach allows tuning the fraction of the CNN that will be executed on each platform based on their capabilities.

Despite their advantages, existing device-cloud collaborative inference solutions suffer from a set of limitations. First, similar to cloud execution, the QoE is greatly affected by the network conditions as execution can fail catastrophically when the link is severely deteriorated. This lack of network fault tolerance also prevents the use of more cost-efficient cloud solutions, e.g. using ephemeral

SPINN: 设备和云上神经网络的协同渐进推理

Stefanos Laskaridis^{†*}、Stylianos I. Venieris^{†*}、
Mario Almeida^{†*}、Ilias Leontiadis^{†*}、Nicholas D. Lane^{†,‡} 剑桥三星人工智能中心*剑桥大学

* Indicates equal contribution. {stefanos.l,s.venieris,mario.a,i.leontiadis,nic.lane}@samsung.com

抽象的

尽管卷积神经网络（CNN）在移动应用中的使用量激增，但由于现代 CNN 的过度计算需求和部署设备的多样性不断增加，在移动设备上统一维持高性能推理一直难以实现。一种流行的替代方案是将 CNN 处理卸载到强大的基于云的服务器。然而，通过依靠云来产生输出，新兴的关键任务和高移动性应用程序（例如无人机避障或交互式应用程序）可能会受到动态连接条件和云的不确定可用性的影响。在本文中，我们提出了 SPINN，这是一种分布式推理系统，它采用协同设备云计算和渐进式推理方法，在不同的设置中提供快速、鲁棒的 CNN 推理。所提出的系统引入了一种新颖的调度程序，可以在运行时共同优化提前退出策略和 CNN 分割，以适应动态条件并满足用户定义的服务级别要求。定量评估表明，SPINN 在不同网络条件下实现的吞吐量比最先进的协作推理同类产品高出 2x，将服务器成本降低了 6.8x，在延迟限制下将准确性提高了 20.7%，同时在不确定的连接条件下提供稳健的操作，并且与以云为中心的执行相比显著节省了能源。

CCS 概念

- 计算方法 → 分布式计算方法；
 - 以人为本的计算 → 无处不在的移动计算。

关键词

深度神经网络、分布式系统、移动计算

1 简介

随着 CNN 驱动的应用程序范围迅速扩大，它们在移动平台上的部署提出了重大挑战。现代 CNN [20, 68] 的计算需求过多，阻碍了它们在资源受限的移动设备中的广泛采用。此外，新兴的面向用户[80]和关键任务[37,42,66]的CNN应用程序需要低延迟处理，以确保高质量的体验（QoE）[4]和安全性[11]。

鉴于最近在消费设备中集成强大的片上系统（SoC）的趋势 [2,25,78]，直接在设备上执行 CNN 已成为可能（图 1 - 左上）。尽管如此，

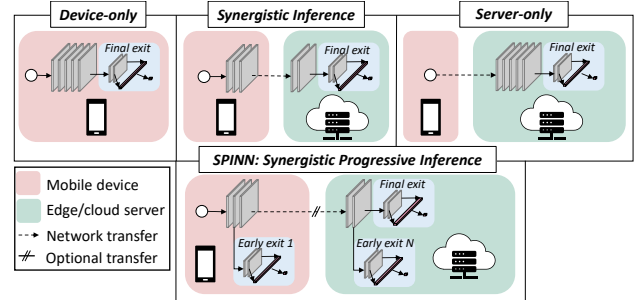


图 1: 现有方法与 SPINN。

虽然旗舰设备可以支持 CNN 工作负载的性能要求，但当前的情况仍然非常多样化，包括上一代和低端模型 [80]。在这种情况下，功能较弱的低层设备难以持续满足应用程序级性能需求 [2]。

作为替代方案，服务提供商通常采用以云为中心的解决方案（图 1 - 右上角）。通过此设置，移动设备收集的输入将传输到远程服务器，以使用强大的加速器执行 CNN 推理 [3,6,12,19,31,32]。然而，这种额外的计算能力是有代价的。首先，云执行高度依赖于动态网络条件，当通信通道性能下降时，性能会急剧下降。其次，托管能够加速机器学习任务的资源需要付出高昂的成本[40]。此外，虽然公共云提供商提供弹性成本扩展，但也存在隐私和安全问题[64]。

为了解决这些限制，最近的一项工作 [22,34,46] 提出了设备和云之间的协作以进行 CNN 推理（图 1 - 顶部中心）。此类方案通常将 CNN 视为计算图，并将其在设备和云之间进行划分。在运行时，客户端执行模型的第一部分并将中间结果传输到远程服务器。服务器继续执行模型并将最终结果返回给设备。总体而言，这种方法允许根据每个平台的功能调整将在每个平台上执行的 CNN 部分。

尽管具有优势，现有的端云协作推理解决方案仍存在一系列限制。首先，与云执行类似，QoE 很大程度上受网络状况影响，因为当链路严重恶化时，执行可能会发生灾难性失败。网络容错能力的缺乏也阻碍了更具成本效益的云解决方案的使用，例如使用短暂的

spare cloud resources at a fraction of the price.¹ Furthermore, CNNs are increasingly deployed in applications with stringent demands across multiple dimensions (e.g. target latency, throughput and accuracy, or device and cloud costs).² Existing collaborative methods cannot sufficiently meet these requirements.

To this end, we present SPINN, a distributed system that enables robust CNN inference in highly dynamic environments, while meeting multi-objective application-level requirements (SLAs). This is accomplished through a novel scheduler that takes advantage of progressive inference; a mechanism that allows the system to exit early at different parts of the CNN during inference, based on the input complexity (Figure 1 - bottom). The scheduler optimises the overall execution by jointly tuning both the split point selection and the early-exit policy at run time to sustain high performance and meet the application SLAs under fluctuating resources (e.g. network speed, device/server load). The guarantee of a local early exit renders server availability non-critical and enables robust operation even under uncertain connectivity. Overall, this work makes the following key contributions:

- A progressive inference mechanism that enables the fast and reliable execution of CNN inference across device and cloud. Concretely, on top of existing early-exit designs, we propose an early-exit-aware cancellation mechanism that allows the interruption of the (local/remote) inference when having a confident early prediction, thus minimising redundant computation and transfers during inference. Simultaneously, reflecting on the uncertain connectivity of mobile devices we design an early-exit scheme with robust execution in mind, even under severe connectivity disruption or cloud unavailability. By carefully placing the early exits in the backbone network and allowing for graceful fallback to locally available results, we guarantee the responsiveness and reliability of the system and overcome limitations of existing offloading systems.
- A CNN-specific packing mechanism that exploits the reduced-precision resilience and sparsity of CNN workloads to minimise transfer overhead. Our communication optimiser combines a lossless and an accuracy-aware lossy compression component which exposes previously unattainable designs for collaborative inference, while not sacrificing the end accuracy of the system.
- An SLA- and condition-aware scheduler that co-optimises i) the early-exit policy of progressive CNNs and ii) their partitioning between device and cloud at run time. The proposed scheduler employs a multi-objective framework to capture the user-defined importance of multiple performance metrics and translate them into SLAs. Moreover, by surveilling the volatile network conditions and resources load at run time, the scheduler dynamically selects the configuration that yields the highest performance by taking into account contextual runtime information and feedback from previous executions.

2 BACKGROUND AND RELATED WORK

To optimise the execution of CNN workloads, several solutions have been proposed, from compiler [1, 30, 65] and runtime optimisations [36, 43, 49] to custom cloud [7, 19, 34] and accelerator designs [75, 79]. While these works target a single model with

device- or cloud-only execution, the increased computational capabilities of client devices [2, 25] have led to schemes that maximise performance via device-cloud synergy. Next, we outline significant work in this direction and visit approximate computing alternatives which exploit accuracy-latency trade-offs during inference.

Approximate Inference. In applications that can tolerate some accuracy drop, a line of work [9, 18, 45] exploits the accuracy-latency trade-off through various techniques. In particular, NestDNN [9] employs a multi-capacity model that incorporates multiple descendant (i.e. pruned) models to expose an accuracy-complexity trade-off mechanism. However, such models cannot be natively split between device and cloud. On the other hand, model selection systems [18, 45] employ multiple variants of a single model (e.g. quantised, pruned) with different accuracy-latency trade-offs. At run time, they choose the most appropriate variant based on the application requirements and determine where it will be executed. Similarly, classifier cascades [21, 33, 38, 39, 71] require multiple models to obtain performance gains. Despite the advantages of both, using multiple models adds substantial overhead in terms of maintenance, training and deployment.

Progressive Inference Networks. A growing body of work from both the research [23, 35, 72, 81, 84] and industry communities [55, 74] has proposed transforming a given model into a progressive inference network by introducing intermediate exits throughout its depth. By exploiting the different complexity of incoming samples, easier examples can early-exit and save on further computations. So far, existing works have mainly explored the hand-crafted design of early-exit architectures (MSDNet [23], SCAN [84]), the platform- and SLA-agnostic derivation of early-exit networks from generic models (BranchyNet [72], SDN [35]) or the hardware-aware deployment of such networks (HAPI [44]). Despite the recent progress, these techniques have not capitalised upon the unique potential of such models to yield high mobile performance through distributed execution and app-tailored early-exiting. In this context, SPINN is the first progressive inference approach equipped with a principled method of selectively splitting execution between device and server, while also tuning the early-exit policy, enabling high performance across dynamic settings.

Device-Cloud Synergy for CNN Inference. To achieve efficient CNN processing, several works have explored collaborative computation over device, edge and cloud. One of the most prominent pieces of work, Neurosurgeon [34], partitions the CNN between a device-mapped *head* and a cloud-mapped *tail* and selects a single split point based on the device and cloud load as well as the network conditions. Similarly, DADS [22] tackles CNN offloading, but from a scheduler-centric standpoint, with the aim to yield the optimal partitioning scheme in the case of high and low server load. However, both systems only optimise for single-criterion objectives (latency or energy consumption), they lack support for app-specific SLAs, and suffer catastrophically when the remote server is unavailable. With a focus on data transfer, JALAD [47] incorporates lossy compression to minimise the offload transmission overhead. Nevertheless, to yield high performance, the proposed system sacrifices substantial accuracy (i.e. >5%). JointDNN [8] modelled CNN offloading as a graph split problem, but targets only

¹AWS Spot Instances – <https://aws.amazon.com/ec2/spot/>.

²Typically expressed as service-level agreements (SLAs).

以极低的价格节省云资源。¹此外, CNN 越来越多地部署在多个维度(例如目标延迟、吞吐量和准确性,或设备和云成本)具有严格要求的应用中。²现有的协作方法无法充分满足这些要求。

为此,我们推出了 SPINN,这是一个分布式系统,可以在高度动态的环境中实现稳健的 CNN 推理,同时满足多目标应用程序级要求(SLA)。这是通过利用渐进推理的新颖调度程序来实现的;一种机制,允许系统根据输入复杂性在推理过程中提前退出 CNN 的不同部分(图 1 - 底部)。调度程序通过在运行时联合调整分割点选择和提前退出策略来优化整体执行,以维持高性能并满足资源波动(例如网络速度、设备/服务器负载)下的应用程序 SLA。本地提前退出的保证使服务器可用性变得不那么重要,并且即使在连接不确定的情况下也能实现稳健的操作。总的来说,这项工作做出了以下主要贡献:

- 渐进式推理机制,可跨设备和云快速可靠地执行 CNN 推理。具体来说,在现有的提前退出设计之上,我们提出了一种提前退出感知的取消机制,该机制允许在有置信的早期预测时中断(本地/远程)推理,从而最大限度地减少推理过程中的冗余计算和传输。同时,考虑到移动设备连接的不确定性,我们设计了一种提前退出方案,即使在严重的连接中断或云不可用的情况下,也具有强大的执行能力。通过仔细地将早期出口放置在骨干网络中并允许优雅地回退到本地可用的结果,我们保证了系统的响应能力和可靠性,并克服了现有卸载系统的限制。
- CNN 特定的打包机制,利用 CNN 工作负载的精度降低弹性和稀疏性来最大限度地减少传输开销。我们的通信优化器结合了无损和精度感知的有损压缩组件,该组件公开了以前无法实现的协作推理设计,同时不牺牲系统的最终精度。
- SLA 和条件感知调度程序,可共同优化 i) 渐进式 CNN 的提前退出策略和 ii) 在运行时设备和云之间的分区。所提出的调度程序采用多目标框架来捕获用户定义的多性能指标的重要性并将其转换为 SLA。此外,通过监视运行时不稳定的网络条件和资源负载,调度程序会考虑上下文运行时信息和先前执行的反馈,动态选择产生最高性能的配置。

2 背景及相关工作

为了优化 CNN 工作负载的执行,人们提出了几种解决方案,从编译器 [1, 30, 65] 和运行时优化 [36, 43, 49] 到定制云 [7, 19, 34] 和加速器设计 [75, 79]。虽然这些作品针对的是单一模型

仅设备或仅云执行,客户端设备[2, 25]计算能力的增强导致了通过设备云协同最大化性能的方案。接下来,我们概述了这个方向的重要工作,并访问了在推理过程中利用准确性与延迟权衡的近似计算替代方案。

近似推论。在可以容忍一定精度下降的应用程序中,一系列工作 [9,18,45] 通过各种技术来利用精度与延迟之间的权衡。特别是, NestDNN [9] 采用了一种多容量模型,该模型结合了多个后代(即修剪的)模型来公开准确性-复杂性权衡机制。然而,此类模型无法在设备和云之间原生分割。另一方面,模型选择系统[18, 45]采用单个模型的多个变体(例如量化、修剪),并具有不同的精度-延迟权衡。在运行时,他们根据应用程序要求选择最合适的变体并确定其执行位置。类似地,分类器级联[21、33、38、39、71]需要多个模型才能获得性能提升。尽管两者都有优点,但使用多个模型会增加维护、培训和部署方面的大量开销。

渐进推理网络。研究 [23, 35, 72, 81, 84] 和行业社区 [55, 74] 越来越多的工作提出,通过在整个深度中引入中间出口,将给定模型转变为渐进推理网络。通过利用传入样本的不同复杂性,更简单的示例可以提前退出并节省进一步的计算。到目前为止,现有的工作主要探索了早期退出架构的手工设计 (MSDN et [23], SCAN [84]), 从通用模型 (BranchyNet [72], SDN [35]) 中对早期退出网络进行与平台和 SLA 无关的推导 (BranchyNet [72], SDN [35]) 或此类网络的硬件感知部署 (HAPI [44])。尽管最近取得了进展,但这些技术尚未利用此类模型的独特潜力,通过分布式执行和应用程序定制的提前退出来产生高移动性能。在这种情况下, SPINN 是第一个渐进式推理方法,配备了在设备和服务器之间有选择地分割执行的原则性方法,同时还调整了提前退出策略,从而在动态设置中实现高性能。

用于 CNN 推理的端云协同。为了实现高效的 CNN 处理,一些工作探索了设备、边缘和云上的协作计算。最突出的工作之一是 Neurosurgeon [34], 它将 CNN 划分为设备映射的头部和云映射的尾部,并根据设备和云负载以及网络条件选择单个分割点。类似地, DADS [22] 解决了 CNN 卸载问题,但从以调度程序为中心的角度来看,目的是在服务器负载高和低的情况下产生最佳分区方案。然而,这两个系统都只针对单一标准目标(延迟或能耗)进行优化,缺乏对特定于应用程序的 SLA 的支持,并且当远程服务器不可用时会遭受灾难性的后果。JALAD [47] 专注于数据传输,采用有损压缩来最大限度地减少卸载传输开销。然而,为了获得高性能,所提出的系统牺牲了大量的准确性(即 >5%)。JointDNN [8] 将 CNN 卸载建模为图分割问题,但仅针对目标

¹AWS Spot Instances – <https://aws.amazon.com/ec2/spot/>.

²Typically expressed as service-level agreements (SLAs).

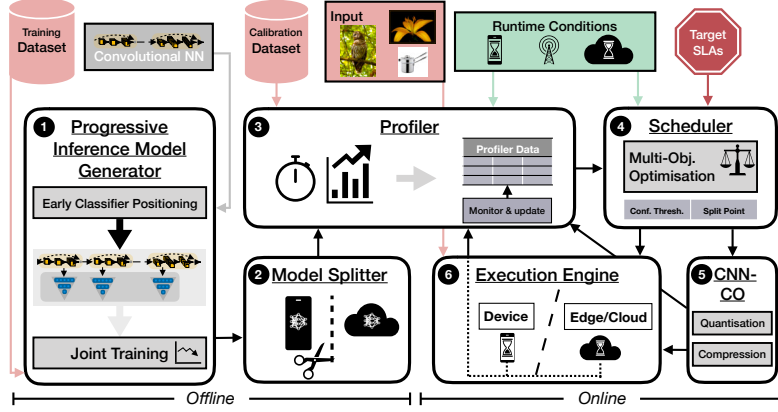


Figure 2: Overview of SPINN’s architecture.

offline scheduling and static environments instead of highly dynamic mobile settings. Contrary to these systems, SPINN introduces a novel scheduler that adapts the execution to the dynamic contextual conditions and jointly tunes the offloading point and early-exit policy to meet the application-level requirements. Moreover, by guaranteeing the presence of a local result, SPINN provides resilience to server disconnections.

Apart from offloading CNNs to a dedicated server, a number of works have focused on tangential problems. IONN [29] tackles a slightly different problem, where instead of preinstalling the CNN model to a remote machine, the client device can offload to any close-by server by transmitting both the incoming data and the model in a shared-nothing setup. Simultaneously, various works [50, 51, 85] have examined the case where the client device can offload to other devices in the local network. Last, [73] also employs cloud-device synergy and progressive inference, but with a very different focus, *i.e.* to perform joint classification from a multi-view, multi-camera standpoint. Its models, though, are statically allocated to devices and its fixed, statically-defined early-exit policy, renders it impractical for dynamic environments.

Offloading Multi-exit Models. Closer to our approach, Edgent [46] proposes a way of merging offloading with multi-exit models. Nonetheless, this work has several limitations. First, the inference workflow disregards data locality and always starts from the cloud. Consequently, inputs are always transmitted, paying an additional transfer cost. Second, early-exit networks are not utilised with progressive inference, *i.e.* inputs do not early-exit based on their complexity. Instead, Edgent tunes the model’s complexity by selecting a *single* intermediary exit for all inputs. Therefore, the end system does not benefit from the variable complexity of inputs. Finally, the system has been evaluated solely on simple models (AlexNet) and datasets (CIFAR-10), less impacted by low-latency or unreliable network conditions. In contrast, SPINN exploits the fact that data already reside on the device to avoid wasteful input transfers, and employs a CNN-tailored technique to compress the offloaded data. Furthermore, not only our scheduler supports additional optimisation objectives, but it also takes advantage of the input’s complexity to exit early, saving resource usage with minimal impact on accuracy.

3 PROPOSED SYSTEM

To remedy the limitations of existing systems, SPINN employs a progressive inference approach to alleviate the hard requirement for reliable device-server communication. The proposed system introduces a scheme of distributing progressive early-exit models across device and server, in which one exit is always present on-device, guaranteeing the availability of a result at all times. Moreover, as early exits along the CNN provide varying levels of accuracy, SPINN casts the acceptable prediction confidence as a tunable parameter to adapt its accuracy-speed trade-off. Alongside, we propose a novel run-time scheduler that jointly tunes the split point and early-exit policy of the progressive model, yielding a deployment tailored to the application performance requirements under dynamic conditions. Next, we present SPINN’s high-level flow followed by a description of its components.

3.1 Overview

SPINN comprises offline components, run once before deployment, and online components, which operate at run time. Figure 2 shows a high-level view of SPINN. Before deployment, SPINN obtains a CNN model and derives a progressive inference network. This is accomplished by introducing early exits along its architecture and jointly training them using the supplied training set (Section 3.2 ①). Next, the model splitter component (Section 3.3 ②) identifies all candidate points in the model where computation can be split between device and cloud. Subsequently, the offline profiler (Section 3.4 ③) calculates the exit-rate behaviour of the generated progressive model as well as the accuracy of each classifier. Moreover, it measures its performance on the client and server, serving as initial inference latency estimates.

At run time, the scheduler (Section 3.5 ④) obtains these initial timings along with the target SLAs and run-time conditions and decides on the optimal split and early-exit policy. Given a split point, the communication optimiser (Section 3.6 ⑤) exploits the CNN’s sparsity and resilience to reduced bitwidth to compress the data transfer and increase the bandwidth utilisation. The execution engine (Section 3.7 ⑥) then orchestrates the distributed inference execution, handling all communication between partitions. Simultaneously, the online profiler monitors the execution across inferences, as well as the contextual factors (*e.g.* network, device/server load) and updates the initial latency estimates. This

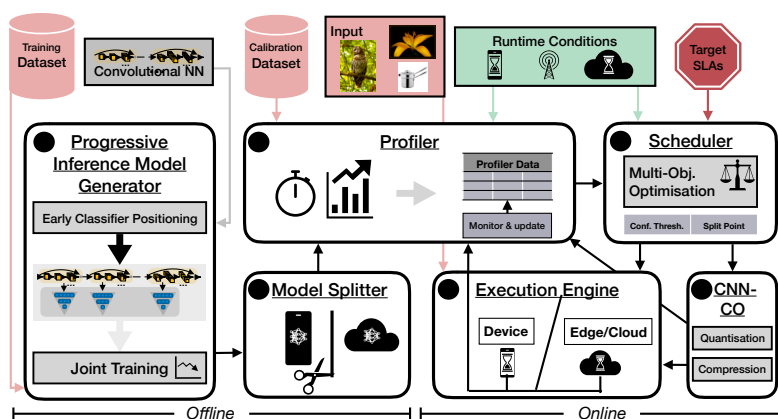


图 2: SPINN 架构概述。

离线调度和静态环境而不是高度动态的移动设置。与这些系统相反，SPINN 引入了一种新颖的调度程序，可以使执行适应动态上下文条件，并联合调整卸载点和提前退出策略以满足应用程序级别的要求。此外，通过保证本地结果的存在，SPINN 提供了对服务器断开连接的恢复能力。

除了将 CNN 卸载到专用服务器之外，许多工作都集中在无关紧要的问题上。IONN [29] 解决了一个稍微不同的问题，客户端设备可以通过在无共享设置中传输输入数据和模型来将 CNN 模型卸载到任何附近的服务器，而不是将 CNN 模型预先安装到远程计算机。同时，各种工作[50,51,85]已经研究了客户端设备可以卸载到本地网络中的其他设备的情况。最后，[73]还采用了云设备协同和渐进推理，但重点非常不同，即从多视图、多摄像机的角度执行联合分类。然而，它的模型是静态分配给设备的，并且其固定的、静态定义的提前退出策略使其对于动态环境来说不切实际。

卸载多出口模型。与我们的方法更接近的是，Edgent [46] 提出了一种将卸载与多出口模型合并的方法。尽管如此，这项工作有一些局限性。首先，推理工作流程不考虑数据局部性，并且始终从云端开始。因此，输入总是被传输，从而支付额外的传输成本。其次，提前退出网络不与渐进推理一起使用，即输入不会根据其复杂性而提前退出。相反，Edgent 通过为所有输入选择一个中间出口来调整模型的复杂性。因此，最终系统不会从输入的可变复杂性中受益。最后，该系统仅在简单模型（AlexNet）和数据集（CIFAR-10）上进行了评估，受低延迟或不可靠网络条件的影响较小。相比之下，SPINN 利用数据已经驻留在设备上的事实来避免浪费的输入传输，并采用 CNN 定制的技术来压缩卸载的数据。此外，我们的调度程序不仅支持额外的优化目标，而且还利用输入的复杂性提前退出，节省资源使用，同时对准确性的影响最小。

3 提议的系统

为了弥补现有系统的局限性，SPINN 采用渐进式推理方法来减轻对可靠设备-服务器通信的硬性要求。所提出的系统引入了一种跨设备和服务器分布渐进式早期退出模型的方案，其中一个退出始终存在于设备上，从而保证结果始终可用。此外，由于 CNN 的早期出口提供了不同程度的准确度，SPINN 将可接受的预测置信度作为可调参数，以适应其准确度与速度的权衡。此外，我们提出了一种新颖的运行时时调度程序，可以联合调整渐进模型的分割点和提前退出策略，从而生成适合动态条件下应用程序性能要求的部署。接下来，我们介绍 SPINN 的高级流程，并对其组件进行描述。

3.1 概述

SPINN 包含离线组件（在部署前运行一次）和在线组件（在运行时运行）。图 2 显示了 SPINN 的高级视图。在部署之前，SPINN 获得 CNN 模型并导出渐进式推理网络。这是通过在其架构中引入早期退出并使用提供的训练集联合训练它们来实现的（第 3.2 节）

。接下来，模型拆分器组件（第 3.3.2 节）识别模型中可以在设备和云之间拆分计算的所有候选点。随后，离线分析器（第 3.4.3 节）计算生成的渐进模型的退出率行为以及每个分类器的准确性。此外，它还测量客户端和服务端上的性能，作为初始推理延迟估计。

在运行时，调度程序（第 3.5.4 节）获取这些初始计时以及目标 SLA 和运行时条件，并决定最佳拆分和提前退出策略。给定分割点，通信优化器（第 3.6.5 节）利用 CNN 的稀疏性和弹性来减少位宽，以压缩数据传输并提高带宽利用率。然后，执行引擎（第 3.7.6 节）协调分布式推理执行，处理分区之间的所有通信。同时，在线分析器监控推理的执行情况以及上下文因素（例如网络、设备/服务器负载）并更新初始延迟估计。这

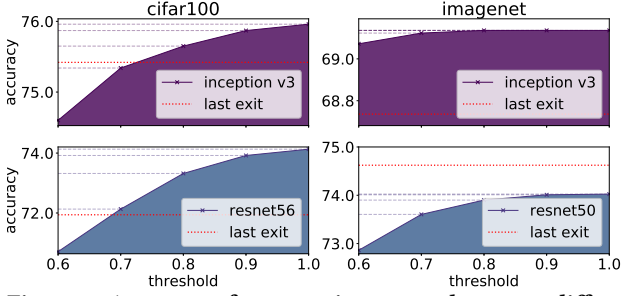


Figure 3: Accuracy of progressive networks across different confidence threshold values.

way, the system can adapt to the rapidly-changing environment, reconfigure its execution and maintain the same QoE.

3.2 Progressive Inference Model Generator

Given a CNN model, SPINN derives a progressive inference network (Figure 2 ①). This process comprises a number of key design decisions: 1) the *number*, *position* and *architecture* of intermediate classifiers (early exits), 2) the *training scheme* and 3) the *early-exit policy*.

Early Exits. We place the intermediate classifiers along the depth of the architecture with equal distance in terms of FLOP count. With this platform-agnostic positioning strategy, we are able to obtain a progressive inference model that supports a wide range of latency budgets while being portable across devices. With respect to their number, we introduce six early exits in order to guarantee their convergence when trained jointly [35, 48], placed at 15%, 30%, ... 90% of the network’s total FLOPs. Last, we treat the architecture of the early exits as an invariant, adopting the design of [23], so that all exits have the same expressivity [59].

Training Scheme. We jointly train all classifiers from scratch and employ the cost function introduced in [35] as follows: $\mathcal{L} = \sum_{i=0}^{N-1} \tau_i * \mathcal{L}_i$ with τ_i starting uniformly at 0.01 and linearly increasing it to a maximum value of C_i , which is the relative position of the classifier in the network ($C_0 = 0.15, C_1 = 0.3, \dots, C_{\text{final}} = 1$). The rationale behind this is to address the problem of “overthinking” [35], where some samples can be correctly classified by early exits while being misclassified deeper on in the network. This scheme requires the fixed placement of early exits prior to the training stage. Despite the inflexibility of this approach to search over different early-exit placements, it yields higher accuracy compared to the two-staged approach of training the main network and the early classifiers in isolation. In terms of training time, the end-to-end early-exit networks can take from $1.2\times$ to $2.5\times$ the time of the original network training, depending on the architecture and number of exits. In fact, the higher training overhead happens when the ratio of $\frac{\text{FLOPs}_{\text{early_classifier}}}{\text{FLOPs}_{\text{original_network}}}$ is higher. However, given that this cost is paid once offline, it is quickly amortised by the runtime latency benefits of early exiting on confident samples.

Early-exit Policy. For the early-exit strategy, we estimate a classifier’s *confidence* for a given input using the top-1 output value of its softmax layer (Eq. (1)) [13]. An input takes the i -th exit if the prediction confidence is higher than a tunable threshold, thr_{conf} (Eq. (2)). The exact value of thr_{conf} provides a trade-off between the latency and accuracy of the progressive model and determines the *early-exit policy*. At run time, SPINN’s scheduler periodically tunes thr_{conf} to customise the execution to the application’s needs.

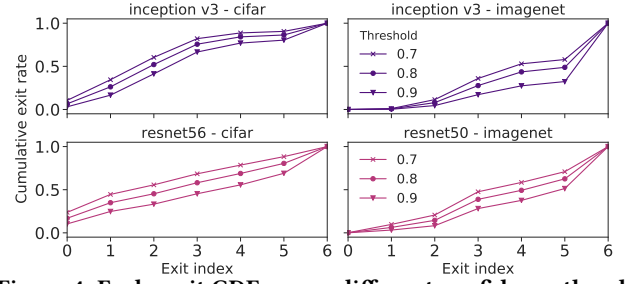


Figure 4: Early-exit CDF across different confidence threshold values.

If none of the classifiers reaches the confidence threshold, the most confident among them is used as the output prediction (Eq. (3)).

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (\text{Softmax of } i\text{-th exit}) \quad (1)$$

$$\arg_i \{ \max_i \{ \text{softmax}_i \} > \text{thr}_{\text{conf}} \} \quad (\text{Check } i\text{-th exit's top-1}) \quad (2)$$

$$\argmax_{j \in \text{classifiers}} \{ \max_i \{ \text{softmax}_i^j \} \} \quad (\text{Return most confident}) \quad (3)$$

where z_i is the output of the final fully-connected layer for the i -th label, K the total number of labels, $j \in [0, 6]$ the classifier index and thr_{conf} the tunable confidence threshold.

Impact of Confidence Threshold. Figure 3 and 4 illustrate the impact of different *early-exit policies* on the accuracy and early-exit rate of progressive models, by varying the confidence threshold (thr_{conf}). Additionally, Figure 3 reports on the accuracy without progressive inference (*i.e.* last exit only, represented by the red dotted line). Note that exiting only at the last exit can lead to lower accuracy than the progressive models for some architectures, a phenomenon that can be attributed to the problem of “overthinking”³.

Based on the figures, we draw two major conclusions that guide the design of SPINN’s scheduler. First, across all networks, we observe a monotonous trend with higher thresholds leading to higher accuracies (Figure 3) while lower ones lead to more samples exiting earlier (Figure 4). This exposes the confidence threshold as a tunable parameter to control accuracy and overall processing time. Second, different networks behave differently, producing *confident* predictions at different exits along the architecture. For example, Inception-v3 on CIFAR-100 can obtain a confident prediction earlier on, whereas ResNet-50 on ImageNet cannot classify robustly from early features only. In this respect, we conclude that optimising the confidence threshold for each CNN explicitly is key for tailoring the deployment to the target requirements.

3.3 Model Splitter

After deriving a progressive model from the original CNN, SPINN aims to split its execution across a client and a server in order to dynamically utilise remote resources as required by the application SLAs. The model splitter (Figure 2 ②) is responsible for 1) defining the potential split points and 2) identifying them automatically in the given CNN.

³“Overthinking” [35] dictates that certain samples that would normally get misclassified by reaching the final classifier of the network if they exit early, they get classified correctly. This leads to small accuracy benefits of progressive inference networks that neither the original model would have (due to early-exiting) nor a single-exit smaller variant (due to late-exiting).

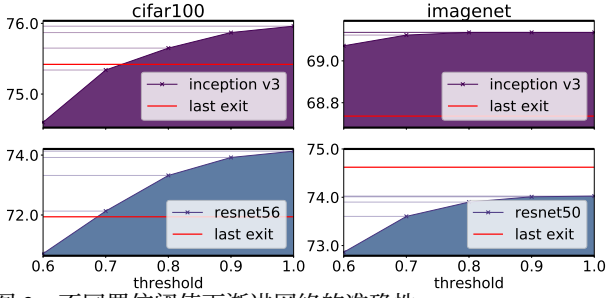


图 3：不同置信阈值下渐进网络的准确性。

这样，系统就可以适应快速变化的环境，重新配置其执行并保持相同的QoE。

3.2 渐进推理模型生成器

给定 CNN 模型，SPINN 派生出渐进式推理网络（图 2 1）。这个过程包括许多关键的设计决策：1）中间分类器（提前退出）的数量、位置和架构，2）训练方案和3）提前退出策略。

提前退出。我们将中间分类器沿着架构的深度放置，在 FLOP 计数方面距离相等。通过这种与平台无关的定位策略，我们能够获得渐进式推理模型，该模型支持广泛的延迟预算，同时可跨设备移植。关于它们的数量，我们引入了 6 个早期退出，以保证它们在联合训练时收敛 [35, 48]，分别为 15%、30%、。。。网络总失败次数的 90%。最后，我们将早期出口的架构视为不变量，采用[23]的设计，使得所有出口具有相同的表现力[59]。

培训计划。我们从头开始联合训练所有分类器，并采用[35]中引入的成本函数，如下所示： $\mathcal{L} = \sum_{i=0}^{N-1} \tau_i * \mathcal{L}_i$ ，其中 τ_i 均匀地从0.01开始，线性增加到 C_i 的最大值， C_i 是分类器在网络中的相对位置（ $C_0 = 0.15$, $C_1 = 0.3$, $C_{final} = 1$ ）。其背后的基本原理是为了解决“过度思考”的问题[35]，即一些样本可以通过早期退出来正确分类，但在网络的更深处却被错误分类。该方案要求在训练阶段之前固定提前退出。尽管这种方法在搜索不同的早期退出位置时缺乏灵活性，但与单独训练主网络和早期分类器的两阶段方法相比，它具有更高的准确性。在训练时间方面，端到端提前退出网络可能需要原始网络训练时间的 1.2x 到 2.5x，具体取决于架构和退出数量。事实上，当 $\frac{FLOPS_{early_classifier}}{FLOPS_{original_network}}$ 的比率较高时，会发生较高的训练开销。然而，考虑到这一成本是在离线时支付的，它很快就会被提前退出可信样本的运行延迟带来的好处所摊销。

提前退出政策。对于提前退出策略，我们使用其 softmax 层的 top-1 输出值来估计分类器对于给定输入的置信度（等式（1））[13]。如果预测置信度高于可调阈值 thr_{conf} （等式（2）），则输入将采用第 i 个退出。 thr_{conf} 的确切值提供了渐进模型的延迟和准确性之间的权衡，并确定了提前退出策略。在运行时，SPINN 的调度程序会定期调整 thr_{conf} 以根据应用程序的需求定制执行。

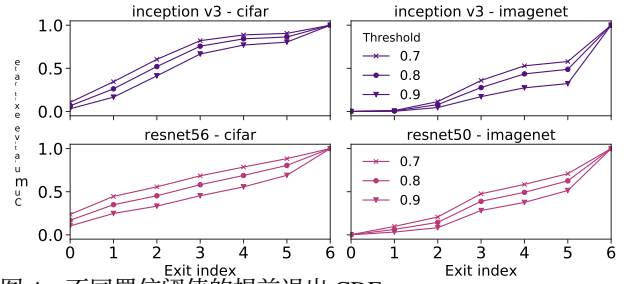


图 4：不同置信阈值的提前退出 CDF。

如果没有一个分类器达到置信度阈值，则将其中最置信度的分类器用作输出预测（式（3））。

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (\text{Softmax of } i\text{-th exit}) \quad (1)$$

$$\arg_i \{ \max_i \{ \text{softmax}_i \} > thr_{conf} \} \quad (\text{Check } i\text{-th exit's top-1}) \quad (2)$$

$$\argmax_{j \in \text{classifiers}} \{ \max_i \{ \text{softmax}_i^j \} \} \quad (\text{Return most confident}) \quad (3)$$

其中 z_i 是第 i 个标签的最终全连接层的输出， K 是标签总数， $j \in [0, 6]$ 是分类器索引， thr_{conf} 是可调置信度阈值。

置信阈值的影响。图 3 和图 4 通过改变置信度阈值 (thr_{conf}) 说明了不同提前退出策略对渐进模型的准确性和提前退出率的影响。此外，图 3 报告了没有渐进推理的准确性（即仅最后一次退出，由红色虚线表示）。请注意，对于某些架构，仅在最后一个出口处退出可能会导致精度低于渐进式模型，这种现象可归因于“过度思考”³的问题。

根据这些图，我们得出两个主要结论来指导 SPINN 调度程序的设计。首先，在所有网络中，我们观察到一个单调的趋势，较高的阈值导致较高的准确度（图 3），而较低的阈值导致更多样本提前退出（图 4）。这将置信阈值公开为可调节参数，以控制准确性和总体处理时间。其次，不同的网络表现不同，在架构的不同出口处产生可信的预测。例如，CIFAR-100 上的 Inception-v3 可以提前获得置信度预测，而 ImageNet 上的 ResNet-50 无法仅根据早期特征进行鲁棒分类。在这方面，我们得出的结论是，明确优化每个 CNN 的置信度阈值是根据目标要求定制部署的关键。

3.3 模型分配器

从原始 CNN 派生出渐进式模型后，SPINN 的目标是将其执行拆分到客户端和服务端之间，以便根据应用程序 SLA 的要求动态利用远程资源。模型分割器（图 2 2）负责 1）定义潜在的分割点，2）在给定的 CNN 中自动识别它们。

³“Overthinking” [35] dictates that certain samples that would normally get misclassified by reaching the final classifier of the network if they exit early, they get classified correctly. This leads to small accuracy benefits of progressive inference networks that neither the original model would have (due to early-exiting) nor a single-exit smaller variant (due to late-exiting).

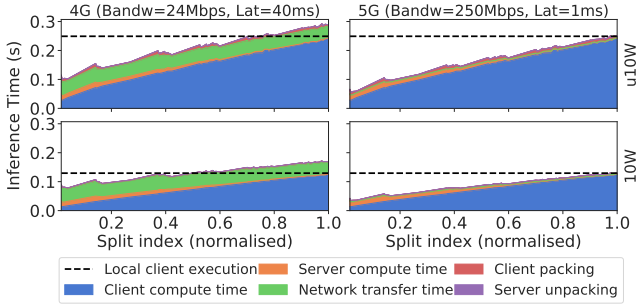


Figure 5: ResNet-56 inference times for different network conditions (4G and 5G) and client device compute capabilities (Jetson 10W and u10W), when offloading to the cloud without early exits.

Split Point Decision Space. CNNs typically consist of a sequence of layers, organised in a feed-forward topology. SPINN adopts a partition scheme which allows splitting the model along its depth at layer granularity. For a CNN with N_L layers, there are $N_L - 1$ candidate points, leading to $2^{N_L - 1}$ possible partitions. To reduce the search space and minimise the number of transmissions across the network, we make two key observations. First, since CNN final outputs are rather small, once execution is offloaded to the powerful remote server, there is no gain in having two or more split points as this would incur in extra communication costs. Second, different layer splits have varying transmission costs and compression potential. For example, activation layers such as ReLU [54] cap negative values to zero, which means that their output becomes more compressible [56, 60, 82] and they can be more efficiently transferred by SPINN’s *communication optimiser* (Section 3.6). Therefore, while SPINN’s design supports an arbitrary number of split points and layers, in this work, we allow one split point per CNN and reduce the candidate split points to ReLU layers.

Automatic Split Point Identification. To automatically detect all candidate split points in the given CNN, the model splitter employs a dynamic analysis approach. This is performed by first constructing the execution graph of the model in the target framework (e.g. *PyTorch*), identifying all split points and the associated dependencies, and then applying SPINN’s partitioning scheme to yield the final pruned split point space. The resulting set of points defines the allowed partitions that can be selected by the scheduler (Section 3.5).

Impact of Split Point. To investigate how the split point selection affects the overall latency, we run multiple CNN splits between a Nvidia Jetson Xavier client and a server (experimental setup detailed in Section 4). Figure 5 shows the breakdown of ResNet-56’s inference times with CIFAR-100 over distinct network conditions and client compute capabilities - u10W and 10W. Packing refers to the runtime of the communication optimiser module, detailed in Section 3.6.

Based on the figure, we make two observations. First, different split points yield varying trade-offs in client, server and transfer time. For example, earlier splits execute more on the server, while later ones execute more on-device but often with smaller transfer

requirements⁴. This indicates that SPINN’s scheduler can selectively choose one to minimise any given runtime (e.g. device, server or transfer), as required by the user. Second, dynamic conditions such as the connectivity, and the device compute capabilities play an important role in shaping the split point latency characteristics illustrated in Figure 5. For example, a lower-end client (u10W) or a loaded server would require longer to execute its allocated split, while low bandwidth can increase the transfer time. This indicates that it is hard to statically identify the best split point and highlights the need for an informed partitioning that adapts to the environmental conditions in order to meet the application-level performance requirements.

3.4 Profiler

Given the varying trade-offs of different split points and confidence thresholds, SPINN considers the client and server load, the networking conditions and the expected accuracy in order to select the most suitable configuration. To estimate this set of parameters, the *profiler* (Figure 2 ③) operates in two stages: i) offline and ii) run-time.

Offline stage: In the offline phase, the profiler performs two kinds of measurements, *device-independent* and *device-specific*. The former include CNN-specific metrics, such as 1) the size of data to be transmitted for each candidate split and 2) the average accuracy of the progressive CNN for different confidence thresholds. These are measured only once prior to deployment. Next, the profiler needs to obtain latencies estimates that are specific to each device. To this end, the profiler measures the average execution time per layer by passing the CNN through a calibration set – sampled from the validation set of the target task. The results serve as the initial latency and throughput estimates.

Run-time stage: At run time, the profiler refines its offline estimates by regularly monitoring the device and server load, as well as the connectivity conditions. To keep the profiler lightweight, instead of adopting a more accurate but prohibitively expensive estimator, we employ a 2-staged linear model to estimate the inference latency on-device.

In the first step, the profiler measures the actual on-device execution time up to the split point s , denoted by $T_{(s)}^{\text{real}}$ during each inference. Next, it calculates a latency scaling factor SF as the ratio between the actual time and the offline latency estimate up to the split s , i.e. $SF = \frac{T_{(s)}^{\text{real}}}{T_{(s)}^{\text{offline}}}$. As a second step, the profiler treats the scaling factor as an indicator of the load of the client device, and uses it to estimate the latency of all other candidate splits. Thus, the latency of a different split s' is estimated as $SF \cdot T_{(s')}^{\text{offline}}$.

Similarly, to assess the server load, the remote endpoint’s compute latency is optionally communicated back to the device, piggy-backed with the CNN response when offloading. If the server does not communicate back latencies for preserving the privacy of the provider, these can be coarsely estimated as $T_{(s)}^{\text{server}} = T_{(s,e)}^{\text{response}} - \left(L + \frac{D_{\text{response}}}{B}\right)$, where $T_{(s)}^{\text{response}}$ is the total time for the server to respond with the result for split point s and exit e , D_{response} is the size of transferred data and B , L are the instantaneous network

⁴Note that independently of the amount of transmitted data, there is always a network latency overhead that must be amortised, which in the case of 4G, is quite significant.

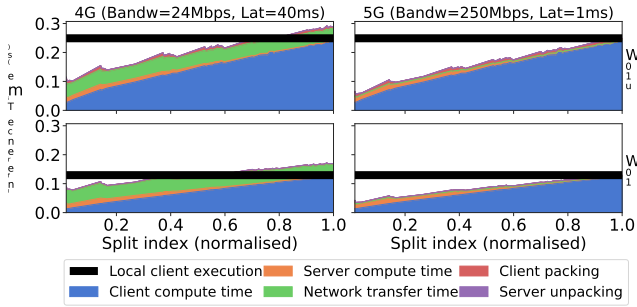


图 5: 在不提前退出的情况下卸载到云端时, 不同网络条件 (4G 和 5G) 和客户端设备计算能力 (Jetson 10W 和 u10W) 的 ResNet-56 推理时间。

分裂点决策空间。CNN 通常由一系列层组成, 以前馈拓扑结构组织。SPINN 采用一种分区方案, 允许以层粒度沿深度分割模型。对于具有 N_L 层的 CNN, 有 $N_L - 1$ 个候选点, 导致 $2^{N_L - 1} - 1$ 个可能的分区。为了减少搜索空间并最大限度地减少网络传输数量, 我们做了两个关键的观察。首先, 由于 CNN 最终输出相当小, 一旦执行被卸载到强大的远程服务器, 拥有两个或更多分割点就没有任何好处, 因为这会产生额外的通信成本。其次, 不同的层分割具有不同的传输成本和压缩潜力。例如, ReLU [54] 等激活层将负值限制为零, 这意味着它们的输出变得更可压缩 [56,60,82], 并且可以通过 SPINN 的通信优化器更有效地传输它们 (第 3.6 节)。因此, 虽然 SPINN 的设计支持任意数量的分割点和层, 但在这项工作中, 我们允许每个 CNN 有一个分割点, 并将候选分割点减少到 ReLU 层。

自动分割点识别。为了自动检测给定 CNN 中的所有候选分割点, 模型分割器采用动态分析方法。这是通过首先在目标框架 (例如 PyTorch) 中构建模型的执行图, 识别所有分割点和相关的依赖关系, 然后应用 SPINN 的分区方案来生成最终的修剪分割点空间来执行的。结果点集定义了调度程序可以选择的允许分区 (第 3.5 节)。

分割点的影响。为了研究分割点选择如何影响整体延迟, 我们在 Nvidia Jetson Xavier 客户端和服务端之间运行多个 CNN 分割 (第 4 节详细介绍了实验设置)。图 5 显示了在不同网络条件和客户端计算能力 (u10W 和 10W) 下 ResNet-56 与 CIFAR-100 的推理时间细分。打包是指通信优化器模块的运行时间, 详细信息请参见第 3.6 节。

根据该图, 我们得出两个观察结果。首先, 不同的分割点会在客户端、服务器和传输时间方面产生不同的权衡。例如, 较早的分片在服务器上执行更多, 而较晚的分片在设备上执行更多, 但通常传输量较小

要求 4. 这表明 SPINN 的调度程序可以根据用户的要求选择性地选择一个来最小化任何给定的运行时间 (例如设备、服务器或传输)。其次, 连接性和设备计算能力等动态条件在塑造图 5 所示的分割点延迟特性方面发挥着重要作用。例如, 低端客户端 (u10W) 或加载的服务器将需要更长的时间来执行其分配的分割, 而低带宽会增加传输时间。这表明静态地识别最佳分割点是很困难的, 并且凸显了需要适应环境条件的明智分区以满足应用程序级性能要求。

3.4 分析器

考虑到不同分割点和置信度阈值的不同权衡, SPINN 会考虑客户端和服务端负载、网络条件和预期精度, 以选择最合适的配置。为了估计这组参数, 分析器 (图 2-3) 分两个阶段运行: i) 离线阶段和 ii) 运行时阶段。

离线阶段: 在离线阶段, 分析器执行两种测量: 与设备无关的测量和特定于设备的测量。前者包括 CNN 特定的指标, 例如 1) 每个候选分割要传输的数据大小, 以及 2) 不同置信度阈值下渐进式 CNN 的平均准确度。这些仅在部署前测量一次。接下来, 分析器需要获取特定于每个设备的延迟估计。为此, 分析器通过使 CNN 通过校准集 (从目标任务的验证集采样) 来测量每层的平均执行时间。结果作为初始延迟和吞吐量估计。

运行时阶段: 在运行时, 分析器通过定期监视设备和服务端负载以及连接条件来完善其离线估计。为了保持分析器的轻量级, 我们没有采用更准确但昂贵的估计器, 而是采用两阶段线性模型来估计设备上的推理延迟。

第一步, 分析器测量到分割点 s 的实际设备上执行时间, 在每次推理期间用 $T_{\text{real}}(s)$ 表示。接下来, 它计算延迟缩放因子 SF 作为实际时间与直到分割 s 的离线延迟估计之间的比率, 即 $SF = \frac{T_{\text{real}}(s)}{T_{\text{offline}}(s)}$ 。第二步, 分析器将缩放因子视为客户端设备负载的指标, 并使用它来估计所有其他候选分割的延迟。因此, 不同分割 s' 的延迟估计为 $SF \cdot T_{\text{离线}}(s')$ 。

同样, 为了评估服务器负载, 可以选择将远程端点的计算延迟传回设备, 并在卸载时搭载 CNN 响应。如果服务器没有为了保护提供者的隐私而返回延迟, 则可以将这些延迟粗略估计为 $T_{\text{服务器}}(s) = T_{\text{响应}}(s, e)$

$\left(\frac{L}{B} + \frac{D_{\text{response}}}{B} \right)$, 其中 $T_{\text{response}}(s)$ 是服务器响应分割点 s 和出口 e 的结果的总时间, D_{response} 是传输数据的大小, B 、 L 是瞬时网络

⁴Note that independently of the amount of transmitted data, there is always a network latency overhead that must be amortised, which in the case of 4G, is quite significant.

Algorithm 1: Flow of dynamic scheduler upon invocation

Input: Space of candidate designs Σ
 Prioritised hard constraints $\langle C_1, C_2, \dots, C_n \rangle$
 Prioritised soft targets $\langle O_1, O_2, \dots, O_{|\mathcal{M}|} \rangle$
 Current network conditions $net = \langle L, B \rangle$
 Current device and server loads $l^{\{dev, server\}}$
 Profiler data prf

Output: Highest performing design $\sigma^* = \langle s^*, thr_{conf}^* \rangle$

```

1  $prf \leftarrow \text{UpdateTimings}(prf, net, l^{dev}, l^{server})$ 
2  $\Sigma^{feasible} \leftarrow \Sigma$ 
3  $/* \text{--- Obtain feasible space based on hard constraints ---} */$ 
4 foreach  $C_i \in \langle C_1, C_2, \dots, C_n \rangle$  do
5    $\Sigma^{feasible} \leftarrow \text{RemoveInfeasiblePoints}(prf, C_i, \Sigma^{feasible})$ 
6    $\hookrightarrow \text{VecCompare}(prf, \Sigma^{feasible}(:, M_i), op_i, thr_i) \quad \forall i \in [1, n]$ 
7 end
8  $/* \text{--- Optimise user-defined metrics - Eq. (4) ---} */$ 
9  $\sigma^* \leftarrow \text{OptimiseUserMetrics}(prf, \langle O_1, O_2, \dots, O_{|\mathcal{M}|} \rangle, \Sigma^{feasible})$ 
10  $\hookrightarrow \text{VecMax/Min}(prf, \Sigma^{feasible}(:, M_i), op_i) \quad \forall i \in [1, |\mathcal{M}|]$ 

```

bandwidth and latency respectively. We periodically offload to the server without stopping the local execution to reassess when the server transitions from “overloaded” to “accepting requests”.

To estimate the instantaneous *network transfer* latency, the profiler employs a run-time monitoring mechanism of the bandwidth B and latency L experienced by the device [15]. The overall transfer time is $L + \frac{D(s)}{B}$, where $D(s)$ is the amount of data to be transferred given split s . As the network conditions change, the monitoring module refines its estimates by means of two moving averages: a real-time estimation (L^{rt} , B^{rt}) and a historical moving average (L^{hist} , B^{hist}). The former is updated and used only when transfers have occurred within the last minutes. If no such information exists, the historical estimates for the same network type are used.

3.5 Dynamic Scheduler

Given the output of the profiler, the dynamic scheduler (Figure 2 4) is responsible for distributing the computation between device and cloud, and deciding the early-exit policy of the progressive inference network. Its goal is to yield the highest performing configuration that satisfies the app requirements. To enable the support of realistic multi-criteria SLAs, the scheduler incorporates a combination of *hard constraints* (e.g. a strict inference latency deadline of 100 ms) and *soft targets* (e.g. minimise cost on the device side). Internally, we capture these under a multi-objective optimisation (MOO) formulation. The current set of metrics is defined as

$\mathcal{M} = \{\text{latency, throughput, server cost, device cost, accuracy}\}$

In SPINN, we interpret cloud and device cost as the execution time on the respective side. The defined metrics set $\mathcal{M}$, together with the associated constraints, can cover a wide range of use-cases, based on the relative importance between the metrics for the target task. Formally, we define a hard constraint as $C = \langle M, op, thr \rangle$ where $M \in \mathcal{M}$ is a metric, op is an operator, e.g. $\leq$, and thr is a threshold value.

With respect to soft optimisation targets, we define them formally as $O = \langle M, min/max/value \rangle$ where a given metric $M \in \mathcal{M}$ is either maximised, minimised or as close as possible to a desirable value. To enable the user to specify the importance of each metric, we employ a multi-objective lexicographic formulation [52], shown in Eq. (4).

$$\min_{\sigma} M_i(\sigma), \text{ s.t. } M_j(\sigma) \leq M_j(\sigma_j^*) \quad (4)$$

$$j = 1, 2, \dots, i-1, \quad i > 1, \quad i = 1, 2, \dots, |\mathcal{M}|$$

where σ represents a design point, $M_i \in \langle M_1, M_2, \dots, M_{|\mathcal{M}|} \rangle$ is the i -th metric in the ordered tuple of soft targets, i is a metric’s position in the importance sequence and $M_j(\sigma_j^*)$ represents the optimum of the j -th metric, found in the j -th iteration. Under this formulation, the user ranks the metrics in order of importance as required by the target use-case.

Algorithm 1 presents the scheduler’s processing flow. As a first step, the scheduler uses the estimated network latency and bandwidth, and device and server loads to update the profiler parameters (line 1), including the attainable latency and throughput, and device and server cost for each candidate configuration. Next, all infeasible solutions are discarded based on the supplied hard constraints (lines 4-7); given an ordered tuple of prioritised constraints $\langle C_1, C_2, \dots, C_n \rangle$, the scheduler iteratively eliminates all configurations $\sigma = \langle s, thr_{conf} \rangle$ that violate them in the given order, where s and thr_{conf} represent the associated split point and confidence threshold respectively. In case there is no configuration to satisfy all the constraints up to $i+1$, the scheduler adopts a best-effort strategy by keeping the solutions that comply with up to the i -th constraint and treating the remaining $n-i$ constraints as soft targets. Finally, the scheduler performs a lexicographic optimisation of the user-prioritised soft targets (lines 9-10). To determine the highest performing configuration σ^* , the scheduler solves a sequence of $|\mathcal{M}|$ single-objective optimisation problems, *i.e.* one for each $M \in \langle M_1, M_2, \dots, M_{|\mathcal{M}|} \rangle$ (Eq. (4)).

Deployment. Upon deployment, the scheduler is run on the client side, since most relevant information resides on-device. In a multi-client setting, this setup is further reinforced by the fact that each client device decides independently on its offloading parameters. However, to be deployable without throttling the resources of the target mobile platform, the scheduler has to yield low resource utilisation at run time. To this end, we vectorise the comparison, maximisation and minimisation operations (lines 5-6 and 9-10 in Algorithm 1) to utilise the SIMD instructions of the target mobile CPU (e.g. the NEON instructions on ARM-based cores) and minimise the scheduler’s runtime.

At run time, although the overhead of the scheduler is relatively low, SPINN only re-evaluates the candidate configurations when the outputs of the profiler change by more than a predefined threshold. For highly transient workloads, we can switch from a moving average to an exponential back-off threshold model for mitigating too many scheduler calls. The scheduler overhead and the tuning of the invocation frequency is discussed in Section 4.3.2.

The server – or HA proxy⁵ [70] in multi-server architectures – can admit and schedule requests on the remote side to balance the workload and minimise inference latency, maximise throughput or minimise the overall cost by dynamically scaling down unused resources. We consider these optimisations cloud-specific and out of the scope of this paper. As a result, in our experiments we account for a single server always spinning and having the model resident to its memory. Nevertheless, in a typical deployment, we would envision a caching proxy serving the models with RDMA to the CPU or GPU of the end server, in a virtualised or serverless environment so as to tackle the cold-start problem [57, 77]. Furthermore, to avoid oscillations (flapping) of computation between the deployed devices

⁵High-Availability proxy for load balancing & fault tolerance in data centres.

Algorithm 1: Flow of dynamic scheduler upon invocation

Input: Space of candidate designs Σ
Prioritised hard constraints $\langle C_1, C_2, \dots, C_n \rangle$
Prioritised soft targets $\langle O_1, O_2, \dots, O_{|M|} \rangle$
Current network conditions $net = \langle L, B \rangle$
Current device and server loads $l^{\{dev, server\}}$
Profiler data prf

Output: Highest performing design $\sigma^* = \langle s^*, thr_{conf}^* \rangle$

```
1  $prf \leftarrow \text{UpdateTimings}(prf, net, l^{dev}, l^{server})$ 
2  $\Sigma^{feasible} \leftarrow \Sigma$ 
3 /* --- Obtain feasible space based on hard constraints --- */
4 foreach  $C_i \in \langle C_1, C_2, \dots, C_n \rangle$  do
5    $\Sigma^{feasible} \leftarrow \text{RemoveInfeasiblePoints}(prf, C_i, \Sigma^{feasible})$ 
6    $\forall \text{VecCompare}(prf, \Sigma^{feasible}(:, M_i), op_i, thr_i) \forall i \in [1, n]$ 
7 end
8 /* --- Optimise user-defined metrics - Eq. (4) --- */
9  $\sigma^* \leftarrow \text{OptimiseUserMetrics}(prf, \langle O_1, O_2, \dots, O_{|M|} \rangle, \Sigma^{feasible})$ 
10  $\forall \text{VecMax/Min}(prf, \Sigma^{feasible}(:, M_i), op_i) \forall i \in [1, |M|]$ 
```

分别是带宽和延迟。我们定期卸载到服务器，而不停止本地执行，以重新评估服务器何时从“过载”过渡到“接受请求”。

为了估计瞬时网络传输延迟，分析器采用了设备所经历的带宽 B 和延迟 L 的运行监控机制 [15]。总传输时间为 $L + D_{(s)}B$ ，其中 $D_{(s)}$ 是给定 $split\ s$ 时要传输的数据量。随着网络状况的变化，监控模块通过两个移动平均值来完善其估计：实时估计 (Lrt, Brt) 和历史移动平均值 ($Lhist, Bhist$)。前者仅在最后几分钟内发生传输时更新和使用。如果不存在此类信息，则使用同一网络类型的历史估计。

3.5 动态调度器

给定分析器的输出，动态调度器（图 2-4）负责在设备和云间分配计算，并决定渐进推理网络的提前退出策略。其目标是产生满足应用程序要求的最高性能配置。为了支持现实的多标准 SLA，调度程序结合了硬约束（例如 100 毫秒的严格推理延迟期限）和软目标（例如最小化设备端的成本）。在内部，我们通过多目标优化（MOO）公式捕获这些。当前的指标集定义为

$M = \{\text{延迟, 吞吐量, 服务器成本, 设备成本, 准确度}\}$ 在 SPINN 中，我们将云和设备成本解释为各自一侧的执行时间。根据目标任务的指标之间的相对重要性，定义的指标集 M 以及相关的约束可以涵盖广泛的用例。形式上，我们定义一个硬约束为 $C = \langle M, op, thr \rangle$ ，其中 $M \in M$ 是一个度量， op 是一个运算符，例如 $\leq$ ， thr 为阈值。

对于软优化目标，我们将它们正式定义为 $O = \langle M, \min/\max/\text{value} \rangle$ ，其中给定的度量 $M \in M$ 要么最大化，要么最小化或尽可能接近期望值。为了使用户能够指定每个指标的重要性，我们采用多目标词典编排 [52]，如等式 1 所示。（4）。

$$\min_{\sigma} M_i(\sigma), \text{ s.t. } M_j(\sigma) \leq M_j(\sigma_j^*) \quad (4)$$
$$j = 1, 2, \dots, i-1, i > 1, i = 1, 2, \dots, |M|$$

其中 σ 表示设计点， $M_i \in \langle M_1, M_2, \dots, M_{|M|} \rangle$ 是软目标有序元组中的第 i 个度量， i 是度量在重要性序列中的位置， $M_j(\sigma_j^*)$ 表示在第 j 次迭代中找到的第 j 个度量的最优值。根据此公式，用户根据目标用例的要求按重要性顺序对指标进行排名。

算法 1 展示了调度程序的处理流程。第一步，调度程序使用估计的网络延迟和带宽以及设备和服务器负载来更新分析器参数（第 1 行），包括可达到的延迟和吞吐量，以及每个候选配置的设备和服务器成本。接下来，根据提供的硬约束，所有不可行的解决方案都被丢弃（第 4-7 行）；给定优先约束 $\langle C_1, C_2, \dots, C_n \rangle$ 的有序元组，调度程序迭代地消除所有违反给定顺序的配置 $\sigma = \langle s, thr_{conf} \rangle$ ，其中 s 和 thr_{conf} 分别表示关联的分割点和置信度阈值。如果没有配置可以满足直到 $i+1$ 的所有约束，调度程序将采用尽力而为的策略，保留符合第 i 个约束的解决方案，并将剩余的 $n-i$ 约束视为软目标。最后，调度程序对用户优先的软目标执行字典式优化（第 9-10 行）。为了确定最高性能的配置 σ^* ，调度程序解决了一系列 $|M|$ 单目标优化问题，即每个 $M \in \langle M_1, M_2, \dots, M_{|M|} \rangle$ （方程一个。（4））。

部署。部署后，调度程序在客户端运行，因为大多数相关信息都驻留在设备上。在多客户端设置中，每个客户端设备独立决定其卸载参数这一事实进一步强化了这种设置。然而，为了在不限制目标移动平台资源的情况下进行部署，调度程序必须在运行时产生较低的资源利用率。为此，我们对比较、最大化和最小化操作（算法 1 中的第 5-6 行和第 9-10 行）进行矢量化，以利用目标移动 CPU 的 SIMD 指令（例如基于 ARM 的内核上的 NEON 指令）并最小化调度程序的运行时间。

在运行时，尽管调度器的开销相对较低，但 SPINN 仅在分析器的输出变化超过预定义阈值时才重新评估候选配置。对于高度瞬态的工作负载，我们可以从移动平均值切换到指数回退阈值模型，以减少过多的调度程序调用。调度程序开销和调用频率的调整在第 4.3.2 节中讨论。

服务器（或多服务器架构中的 HA 代理 [570]）可以接受和调度远程端的请求，以平衡工作负载并最小化推理延迟，最大化吞吐量或通过动态缩减未使用的资源来最小化总体成本。我们认为这些优化是特定于云的，超出了本文的范围。因此，在我们的实验中，我们考虑了一台始终旋转并将模型驻留在其内存中的服务器。然而，在典型的部署中，我们会设想在虚拟化或无服务器环境中使用 RDMA 将模型提供给终端服务器的 CPU 或 GPU 的缓存代理，以解决冷启动问题 [57, 77]。此外，为了避免部署的设备之间的计算振荡（抖动）

⁵High-Availability proxy for load balancing & fault tolerance in data centres.

and the available servers, techniques used for data-center traffic flapping are employed [5].

3.6 CNN Communication Optimiser

CNN layers often produce large volumes of intermediate data which come with a high penalty in terms of network transfer. A key enabler in alleviating the communication bottleneck in SPINN is the communication optimiser module (CNN-CO) (Figure 2 ⑤). CNN-CO comprises two stages. In the first stage, we exploit the resilience of CNNs to low-precision representation [14, 16, 28, 38] and lower the data precision from 32-bit floating-point down to 8-bit fixed-point through linear quantisation [28, 53]. By reducing the bitwidth of *only* the data to be transferred, our scheme allows the transfer size to be substantially lower without significant impact on the accuracy of the subsequent classifiers (*i.e.* <0.65 percentage point drop across all exits of the examined CNNs). Our scheme differs from both i) *weights-only reduction* [17, 67, 86], which minimises the model size rather than activations’ size and ii) *all-layers quantisation* [14, 16, 24, 28] which requires complex techniques, such as quantisation-aware training [24, 28] or a re-training step [14, 16], to recover the accuracy drop due to the precision reduction across all layers.

The second stage exploits the observation that activation data are amenable to compression. A significant fraction of activations are zero-valued, meaning that they are sparse and highly compressible. As noted by prior works [56, 60, 82], this sparsity of activations is due to the extensive use of the ReLU layer that follows the majority of layers in modern CNNs. In CNN-CO, sparsity is further magnified due to the reduced precision. In this respect, CNN-CO leverages the sparsity of the 8-bit data by means of an LZ4 compressor with bit shuffling.

At run time, SPINN predicts whether the compression cost will outweigh its benefits by comparing the estimated CNN-CO runtime to the transfer time savings. If CNN-CO’s overhead is amortised, SPINN queues offloading requests’ data to the CNN-CO, with dedicated threads for each of the two stages, before transmission. Upon reception at the remote end, the data are decompressed and cast back to the original precision to continue inference. The overhead is shown as *packing* in Figure 5 for non-progressive models.

3.7 Distributed Execution Engine

In popular Deep Learning frameworks, such as *TensorFlow* [1] and *PyTorch* [58], layers are represented by *modules* and data in the form of multi-dimensional matrices, called *tensors*. To split and offload computation, SPINN modifies CNN layer’s operations behind the scenes. To achieve this, it intercepts module and tensor operations by replacing their functions with a custom wrapper using Python’s function decorators.

Figure 6 focuses on an instance of an example ResNet block. SPINN attributes IDs to each layer in a trace-based manner, by executing the network and using the layer’s execution order as a sequence identifier.⁶ SPINN uses these IDs to build an execution graph in the form of a directed acyclic graph (DAG), with nodes representing tensor operations and edges the tensor flows among them [1, 62, 75]. This is then used to detect the dependencies across

⁶Despite the existence of branches, CNN execution tends to be parallelised across data rather than layers. Hence, the numbering is deterministic.

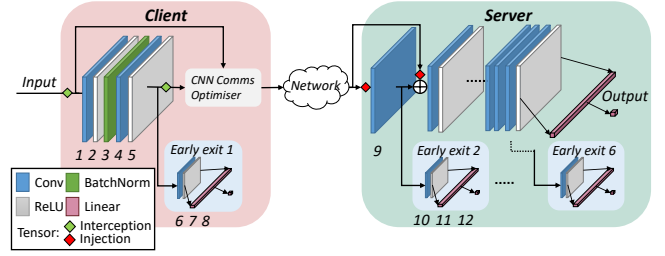


Figure 6: Offloading a progressive ResNet block.

Platform	CPU	Clock Freq.	Memory	GPU
Server	2× Intel Xeon Gold 6130	2.10 GHz	128 GB	GTX1080Ti
Jetson AGX	Carmel ARMv8.2	2.26 GHz	16 GB	512-core Volta

Table 1: Specifications of evaluated platforms.

partitions. To achieve this, Python’s dynamic instance attribute registration is used to taint tensors and monitor their flow through the network. With Figure 6 as a reference, SPINN’s custom wrapper (Figure 2 ⑥) performs the following operations:

Normal execution: When a layer is to be run locally, the wrapper calls the original function it replaced. In Figure 6, layers 1 to 8 execute normally on-device, while layers from 9 until the end execute normally on the server side.

Offload execution: When a layer is selected as a partition point (layer 9), instead of executing the original computation on the client, the wrapper queues an offloading request to be transmitted. This request contains the inputs (layer 5’s output) and subsequent layer dependencies (input of skip connection). Furthermore, the inference and the data transfer are decoupled in two separate threads, to allow for pipelining of data transfer and next frame processing.

Resume execution: Upon receiving an offloading request, the inputs and dependencies are injected in the respective layers (layer 9 and *add* operation) and normal execution is resumed on the remote side. When the server concludes execution, the results are sent back in a parallel thread.

Early exit: When an intermediate classifier (*i.e.* early exit) is executed, the wrapper evaluates its prediction confidence (Eq. (1)). If it is above the provided thr_{conf} , execution terminates early, returning the current prediction (Eq. (2)).

Since the premise of our system is to always have at least one usable result on the client side, we continue the computation on-device even past the split layer, until the next early exit is encountered. Furthermore, to avoid wasteful data transmission and redundant computation, if a client-side early exit covers the latency SLA and satisfies the selected thr_{conf} , the client sends a termination signal to the remote worker to cancel the rest of the inference. If remote execution has not started yet, SPINN does not offload at all.

4 EVALUATION

This section presents the effectiveness of SPINN in significantly improving the performance of mobile CNN inference by examining its core components and comparing with the currently standard device- and cloud-only implementations and state-of-the-art collaborative inference systems.

并采用可用的服务器、用于数据中心流量振荡的技术[5]。

3.6 CNN通信优化器

CNN 层通常会产生大量中间数据，这在网络传输方面会带来很高的损失。缓解 SPINN 中通信瓶颈的关键推动者是通信优化器模块 (CNN-CO) (图 2-5)。CNN-CO 包括两个阶段。在第一阶段，我们利用 CNN 对低精度表示的弹性 [14, 16, 28, 38]，并通过线性量化将数据精度从 32 位浮点降低到 8 位定点 [28, 53]。通过仅减少要传输的数据的位宽，我们的方案允许传输大小显著降低，而不会对后续分类器的准确性产生重大影响（即，在所检查的 CNN 的所有出口上 < 下降了 0.65 个百分点）。我们的方案不同于 i) 仅权重减少 [17, 67, 86]，它最小化了模型大小而不是激活的大小；ii) 全层量化 [14, 16, 24, 28]，它需要复杂的技术，例如量化感知训练 [24, 28] 或重新训练步骤 [14, 16]，以恢复由于所有层的精度降低而导致的精度下降。

第二阶段利用了激活数据易于压缩的观察结果。很大一部分激活是零值的，这意味着它们是稀疏的且高度可压缩的。正如先前的工作 [56, 60, 82] 所指出的，这种激活的稀疏性是由于现代 CNN 中大多数层后面的 ReLU 层的广泛使用所致。在 CNN-CO 中，由于精度降低，稀疏性被进一步放大。在这方面，CNN-CO 通过具有位组功能的 LZ4 压缩器来利用 8 位数据的稀疏性。

在运行时，SPINN 通过将估计的 CNN-CO 运行时间与节省的传输时间进行比较来预测压缩成本是否会超过其好处。如果 CNN-CO 的开销被摊销，SPINN 将在传输之前将请求数据排队卸载到 CNN-CO，并为两个阶段中的每个阶段提供专用线程。在远程端接收到数据后，数据被解压缩并恢复到原始精度以继续推理。对于非渐进式模型，开销如图 5 中的打包所示。

3.7 分布式执行引擎

在流行的深度学习框架中，例如 TensorFlow [1] 和 PyTorch [58]，层由多维矩阵形式的模块和数据表示，称为张量。为了分割和卸载计算，SPINN 在幕后修改了 CNN 层的操作。为了实现这一点，它通过使用 Python 函数装饰器将模块和张量操作替换为自定义包装器来拦截模块和张量操作。

图 6 重点介绍了示例 ResNet 模块的实例。SPINN 通过执行网络并使用层的执行顺序作为序列标识符，以基于跟踪的方式将 ID 分配给每个层。6 SPINN 使用这些 ID 以有向无环图 (DAG) 的形式构建执行图，其中节点表示张量操作，边表示张量在其中流动 [1, 62, 75]。然后用它来检测跨域的依赖关系

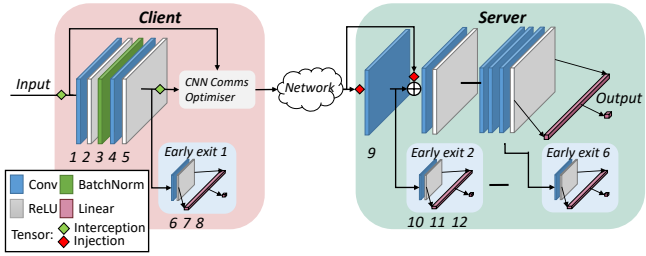


图 6: 卸载渐进式 ResNet 模块。

Platform	CPU	Clock Freq.	Memory	GPU
Server	2× Intel Xeon Gold 6130	2.10 GHz	128 GB	GTX1080Ti
Jetson AGX	Carmel ARMv8.2	2.26 GHz	16 GB	512-core Volta

表 1: 评估平台的规格。

分区。为了实现这一点，Python 的动态实例属性注册用于污染张量并监视它们在网络中的流量。以图 6 为参考，SPINN 的自定义包装器 (图 2.6) 执行以下操作：

正常执行：当一个层要在本地运行时，包装器调用它所替换的原始函数。在图 6 中，第 1 层到第 8 层在设备上正常执行，而从第 9 层到最后的层在服务器端正常执行。

卸载执行：当选择某个层作为分区点 (第 9 层) 时，包装器不会在客户端上执行原始计算，而是将要传输的卸载请求排队。该请求包含输入 (第 5 层的输出) 和后续层依赖项 (跳过连接的输入)。此外，推理和数据传输在两个单独的线程中分离，以允许数据传输和下一帧处理的流水线操作。

恢复执行：收到卸载请求后，输入和依赖项将被注入到各个层 (第 9 层和添加操作)，并在远程端恢复正常执行。当服务器结束执行时，结果将在并行线程中发回。

提前退出：当执行中间分类器 (即提前退出) 时，包装器评估其预测置信度 (等式 (1))。如果它高于提供的 thrconf，则执行提前终止，返回当前预测 (等式 (2))。

由于我们系统的前提是客户端始终至少有一个可用的结果，因此即使经过了分割层，我们仍会在设备上继续计算，直到遇到下一个提前退出。此外，为了避免浪费数据传输和冗余计算，如果客户端提前退出覆盖延迟 SLA 并满足所选的 thrconf，则客户端会向远程工作人员发送终止信号以取消其余的推理。如果远程执行尚未开始，SPINN 根本不会卸载。

4 评价

本节通过检查其核心组件并与当前标准的仅设备和云实现以及最先进的协作推理系统进行比较，介绍了 SPINN 在显着提高移动 CNN 推理性能方面的有效性。

⁶Despite the existence of branches, CNN execution tends to be parallelised across data rather than layers. Hence, the numbering is deterministic.

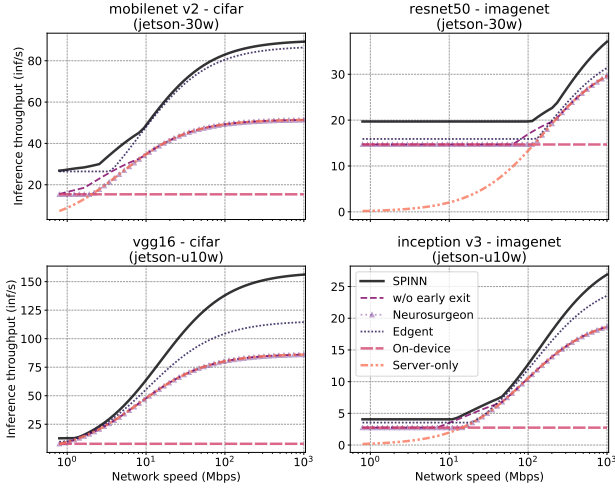


Figure 7: Achieved throughput for various (model, device, dataset) setups vs. network speed.

4.1 Experimental Setup

For our experiments, we used a powerful computer as the server and an Nvidia Jetson Xavier AGX as the client (Table 1). Specifically for Jetson, we tested against three different power profiles to emulate end-devices with different compute capabilities:⁷ 1) 30W (full power), 2) 10W (low power), 3) *underclocked* 10W (u10W). Furthermore, to study the effect of limited computation capacity (e.g. high-load server), we emulated the load by linearly scaling up the CNN computation times on the server side. We simulated the network conditions of offloading by using the average bandwidth and latency across national carriers [26, 27], for 3G, 4G and 5G mobile networks. For local-area connections (Gigabit Ethernet 802.3, Wi-Fi 5 802.11ac), we used the nominal speeds of the protocol. We have developed SPINN on top of *PyTorch* (1.1.0) and experimented with four models, altered from *torchvision* (0.3.0) to include early exits or to reflect the CIFAR-specific architectural changes. We evaluated SPINN using: ResNet-50 and -56 [20], VGG16 [63], MobileNetV2 [61] and Inception-v3 [69]. Unless stated otherwise, each benchmark was run 50 times to obtain the average latency.

Datasets and Training. We evaluated SPINN on two datasets, namely CIFAR-100 [41] and ImageNet (ILSVRC2012) [10]. The former contains 50k training and 10k test images of resolution 32×32, each corresponding to one of 100 labels. The latter is significantly larger, with 1.2m training and 50k test images of 300×300 and 1000 labels. We used the preprocessing steps described in each model’s implementation, such as *scaling* and *cropping* the input image, stochastic image *flip* ($p = 0.5$) and colour channel *normalisation*. After converting these models to progressive early-exit networks, we trained them jointly from scratch end-to-end, with the “overthink” cost function (Section 3.2). We used the authors’ training hyperparameters, except for MobileNetV2, where we utilised SGD with learning rate of 0.05 and cosine learning rate scheduling, due to convergence issues. We trained the networks for 300 epochs on CIFAR-100 and 90 epochs on ImageNet.

⁷We are adjusting the TDP and clock frequency of the CPU and GPU cores, effectively emulating different tiers of devices, ranging from high-end embedded devices to mid-tier smartphones.

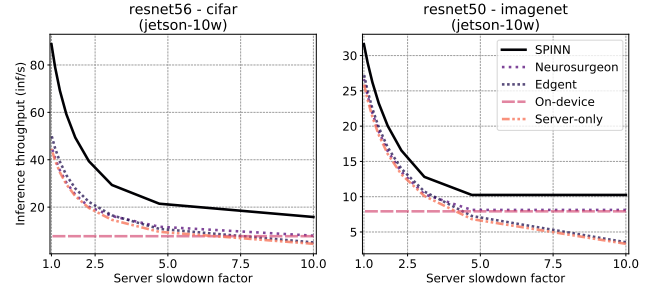


Figure 8: Effect of server slowdown on ResNet.

4.2 Performance Comparison

This subsection presents a performance comparison of SPINN with: 1) the two state-of-the-art CNN offloading systems Neurosurgeon [34] and Edgent [46] (Section 2); 2) the status-quo cloud- and device-only baselines; and 3) a non-progressive ablated variant of SPINN.

4.2.1 Throughput Maximisation. Here, we assess SPINN’s inference throughput across varying network conditions. For these experiments, the SPINN scheduler’s objectives were set to maximise throughput with up to 1 percentage point (pp) tolerance in accuracy drop with respect to the CNN’s last exit.

Figure 7 shows the achieved inference throughput for varying network speeds. On-device execution yields the same throughput independently of the network variation, but is constrained by the processing power of the client device. Server-only execution follows the trajectory of the available bandwidth. Edgent always executes a part of the CNN (up to the exit that satisfies the 1pp accuracy tolerance) irrespective of the network conditions. As a result, it follows a similar trajectory to server-only but achieves higher throughput due to executing only part of the model. Neurosurgeon demonstrates a more polarised behaviour; under constrained connectivity it executes the whole model on-device, whereas as bandwidth increases it switches to offloading all computation as it results in higher throughput. The ablated variant of SPINN (*i.e.* without early exits) largely follows the behaviour of Neurosurgeon at the two extremes of the bandwidth while in the middle range, it is able to achieve higher throughput by offloading earlier due to CNN-CO compressing the transferred data.

The end-to-end performance achieved by SPINN delivers the highest throughput across all setups, achieving a speedup of up to 83% and 52% over Neurosurgeon and Edgent, respectively. This can be attributed to our bandwidth- and data-locality-aware scheduler choices on the early-exit policy and partition point. In low bandwidths, SPINN selects device-only execution, outperforming all other on-device designs due to its early-exiting mechanism, tuned by the scheduler module. In the mid-range, the CNN-CO module enables SPINN to better utilise the available bandwidth and start offloading earlier on, outperforming both Edgent and Neurosurgeon. In high-bandwidth settings, our system surpasses the performance of all other designs by exploiting its optimised early-exiting scheme.

Specifically, compared to Edgent, SPINN takes advantage of the input’s classification difficulty to exit early, whereas the latter only selects an intermediate exit to *uniformly* classify all incoming samples. Moreover, in contrast with Edgent’s strategy to always transmit the input to the remote endpoint, we exploit the fact that data already reside on the device and avoid the wasteful data transfers.

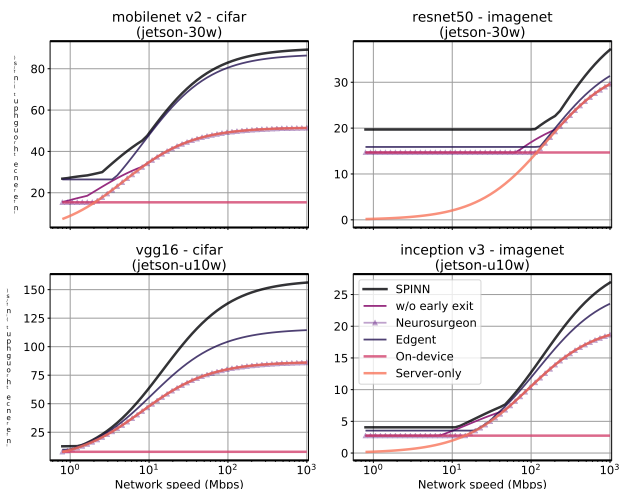


图 7: 各种(模型、设备、数据集)设置所实现的吞吐量与网络速度的关系。

4.1 实验设置

在我们的实验中，我们使用一台功能强大的计算机作为服务器，并使用 Nvidia Jetson Xavier AGX 作为客户端（表 1）。特别是对于 Jetson，我们针对三种不同的功率配置进行了测试，以模拟具有不同计算能力的终端设备：1) 30W（全功率）、2) 10W（低功率）、3) 降频 10W (u10w)。此外，为了研究有限计算能力（例如高负载服务器）的影响，我们通过线性扩展服务器端的 CNN 计算时间来模拟负载。我们使用全国运营商 [26, 27] 的 3G、4G 和 5G 移动网络的平均带宽和延迟来模拟卸载的网络条件。对于本地连接（千兆位以太网 802.3、WiFi-5 802.11ac），我们使用了协议的标称速度。我们在 PyTorch (1.1.0) 之上开发了 SPINN，并尝试了四种模型，这些模型是从 torchvision (0.3.0) 更改而来的，以包括早期退出或反映 CIFAR 特定的架构变化。我们使用 ResNet-50 和 -56 [20]、VGG16 [63]、MobileNetV2 [61] 和 Inception-v3 [69] 评估 SPINN。除非另有说明，每个基准测试运行 50 次以获得平均延迟。

数据集和培训。我们在两个数据集上评估了 SPINN，即 CIFAR-100 [41] 和 ImageNet (ILSVRC2012) [10]。前者包含分辨率为 32×32 的 50k 训练图像和 10k 测试图像，每个图像对应 100 个标签之一。后者明显更大，有 1.2m 训练图像和 50k 测试图像，包含 300×300 和 1000 个标签。我们使用了每个模型实现中描述的预处理步骤，例如缩放和裁剪输入图像、随机图像翻转 ($p = 0.5$) 和颜色通道归一化。将这些模型转换为渐进式早期退出网络后，我们使用“过度思考”成本函数（第 3.2 节）从头开始端到端地联合训练它们。我们使用了作者的训练超参数，但 MobileNetV2 除外，由于收敛问题，我们使用了学习率为 0.05 的 SGD 和余弦学习率调度。我们在 CIFAR-100 上对网络进行了 300 个时期的训练，在 ImageNet 上对网络进行了 90 个时期的训练。

⁷We are adjusting the TDP and clock frequency of the CPU and GPU cores, effectively emulating different tiers of devices, ranging from high-end embedded devices to mid-tier smartphones.

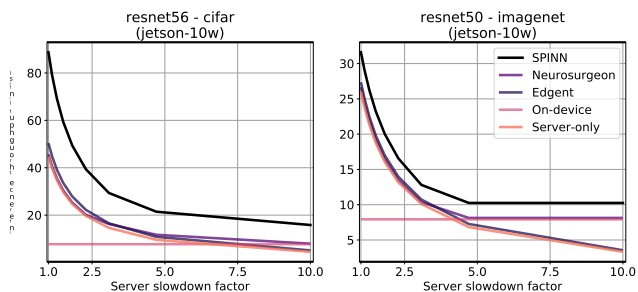


图 8: 服务器速度减慢对 ResNet 的影响。

4.2 性能比较

本小节介绍了 SPINN 与以下各项的性能比较：1) 两个最先进的 CNN 卸载系统 Neurosurgeon [34] 和 Edgent [46]（第 2 节）；2) 仅云和设备的现状基线；3) SPINN 的非渐进式消融变体。

4.2.1 吞吐量最大化。在这里，我们评估了 SPINN 在不同网络条件下的推理吞吐量。对于这些实验，SPINN 调度程序的目标被设置为最大化吞吐量，相对于 CNN 的最后一次退出，精度下降的容忍度高达 1 个百分点 (pp)。

图 7 显示了不同网络速度下实现的推理吞吐量。设备上执行产生相同的吞吐量，与网络变化无关，但受到客户端设备处理能力的限制。仅服务器执行遵循可用带宽的轨迹。无论网络条件如何，Edgent 始终执行 CNN 的一部分（直到满足 1pp 精度容差的出口）。因此，它遵循与仅服务器类似的轨迹，但由于仅执行模型的一部分而实现了更高的吞吐量。神经外科医生表现出更加两极分化的行为；在受限连接下，它在设备上执行整个模型，而随着带宽的增加，它会切换到卸载所有计算，因为这会带来更高的吞吐量。SPINN 的消融变体（即没有提前退出）很大程度上遵循神经外科医生在带宽的两个极端的行为，而在中间范围，由于 CNN-CO 压缩传输的数据，它能够通过更早卸载来实现更高的吞吐量。

SPINN 实现的端到端性能在所有设置中提供了最高的吞吐量，分别比 Neurosurgeon 和 Edgent 提高了 83% 和 52%。这可以归因于我们对提前退出策略和分区点的带宽和数据局部性感知的调度程序选择。在低带宽下，SPINN 选择仅设备执行，由于其提前退出机制（由调度程序模块调整），其性能优于所有其他设备上设计。在中端，CNN-CO 模块使 SPINN 能够更好地利用可用带宽并更早开始卸载，性能优于 Edgent 和 Neurosurgeon。在高带宽设置中，我们的系统通过利用其优化的提前退出方案超越了所有其他设计的性能。

具体来说，与 Edgent 相比，SPINN 利用输入的分类难度提前退出，而后者仅选择中间退出来对所有输入样本进行统一分类。此外，与 Edgent 始终将输入传输到远程端点的策略相比，我们利用数据已经驻留在设备上的事实，避免了浪费的数据传输。

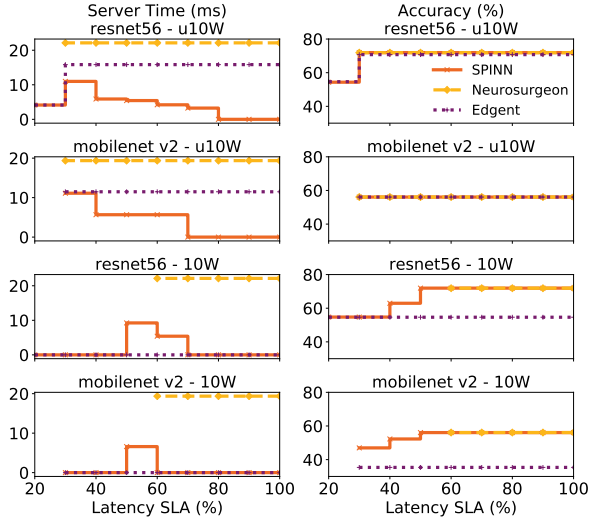


Figure 9: Server time (left) and accuracy (right) of SPINN vs. Neurosurgeon and Edgent for different latency SLAs and client compute power (u10W and 10W). The SLA is expressed as a percentage of the on-device latency.

4.2.2 Server-Load Variation. To investigate the performance of SPINN under various server-side loads, we measured the inference throughput of SPINN against baselines when varying the load of the remote end, with 1pp of accuracy drop tolerance. This is accomplished by linearly scaling the latency of the server execution by a slowdown factor (*i.e.* a factor of 2 means the server is 2× slower). Figure 8 presents the throughput achieved by each approach under various server-side loads, with the Jetson configured at the 10W profile and the network speed in the WiFi-5 range (500 Mbps).

With low server load (left of the x-axis), the examined systems demonstrate a similar trend to the high-bandwidth performance of Figure 7. As the server becomes more loaded (*i.e.* towards the right-hand side), performance deteriorates, except for the case of device-only execution which is invariant to the server load. On the one hand, although its attainable throughput reduces, Neurosurgeon adapts its policy based on the server utilisation and gradually executes a greater fraction of the CNN on the client side. On the other hand, Edgent’s throughput deteriorates more rapidly and even reaches below the device-only execution under high server load, since its run-time mechanism does not consider the varying server load. Instead, by adaptively optimising both the split point and the early-exit policy, SPINN’s scheduler manages to adapt the overall execution based on the server-side load, leading to throughput gains between 1.18-1.99× (1.57× geo. mean) and 1.15-3.09× (1.61× geo. mean) over Neurosurgeon and Edgent respectively.

4.2.3 Case Study: Latency-driven SLAs at minimal server cost. To assess SPINN’s performance under deadlines, we target the scenario where a service provider aims to deploy a CNN-based application that meets strict latency SLAs at maximum accuracy and minimal server-side cost. In this setting, we compare against Neurosurgeon⁸ and Edgent, targeting MobileNetV2 and ResNet-56 over 4G. Figure 9 shows the server computation time and accuracy achieved by each

system for two device profiles with different compute capabilities - u10W and 10W. Latency SLAs are represented as a percentage of the device-only runtime of the original CNN (*i.e.* 20% SLA means that the target is 5× less latency than on-device execution, requiring early exiting and/or server support).

For the low-end device (u10W) and strict latency deadlines, SPINN offloads as much as possible to the cloud as it allows reaching faster a later exit in the network, hence increasing the accuracy. As the SLA loosens (reaching more than 40% of the on-device latency), SPINN starts to gradually execute more and more locally. In contrast, Edgent and Neurosurgeon achieve similar accuracy but with up to 4.9× and 6.8× higher server load. On average across all targets, SPINN reduces Edgent and Neurosurgeon server times by 68.64% and 82.5% (60.3% and 83.6% geo. mean), respectively, due to its flexible multi-objective scheduler. Instead, Neurosurgeon can only optimise for overall latency and cannot trade off accuracy to meet the deadline (*e.g.* for 20% SLA on ResNet-56) while Edgent cannot account for server-time minimisation and accuracy drop constraints.

The situation is different for the more powerful device (10W). With the device being faster, the SLA targets become much stricter. Therefore, we observe that SPINN and Edgent can still meet a latency constraint as low as 20% and 30% of the local execution time for ResNet-56 and MobileNetV2 respectively. In contrast, without progressive inference, it is impossible for Neurosurgeon to achieve inference latency below 60% of on-device execution across both CNNs. In this context, SPINN is able to trade off accuracy in order to meet stricter SLAs, but also improve its attainable accuracy as the latency constraints are relaxed.

For looser latency deadlines (target larger than 50% of the on-device latency), SPINN achieves accuracy gains of 17.3% and 20.7% over Edgent for ResNet-56 and MobileNetV2, respectively. The reason behind this is twofold. First, when offloading, Edgent starts the computation on the server side, increasing the communication latency overhead. Instead, SPINN’s client-to-server offloading strategy and compression significantly reduces the communication latency overhead. Second, due to Edgent’s unnormalised cost function (*i.e.* $\max\left(\frac{1}{Lat} + acc\right)$), the throughput’s reward dominates the accuracy gain, leading to always selecting the first early-exit sub-network and executing it locally. In contrast, SPINN’s scheduler’s multi-criteria design is able to capture accuracy, server time and latency constraints to yield an optimised deployment. Hence, similarly to the slower device, SPINN successfully exploits the server resources to boost accuracy under latency constraints, while it can reach up to pure on-device execution for loose deadlines.

4.3 Runtime Overhead and Efficiency

4.3.1 Deployment Overhead. By evaluating across our examined CNNs and datasets on the CPU of Jetson, the scheduler executes in max 14 ms (11 ms geo. mean). This time includes the cost of reading the profiler parameters, updating the monitored metrics, and searching for and returning the selected configuration. Moreover, SPINN’s memory consumption is in the order of a few KB (*i.e.* <1% of Jetson’s RAM). These costs are amortised over multiple inferences, as the scheduler is invoked only on significant context changes. We discuss the selection of such parameters in the following section.

⁸It should be noted that Neurosurgeon maintains the accuracy of the original CNN.

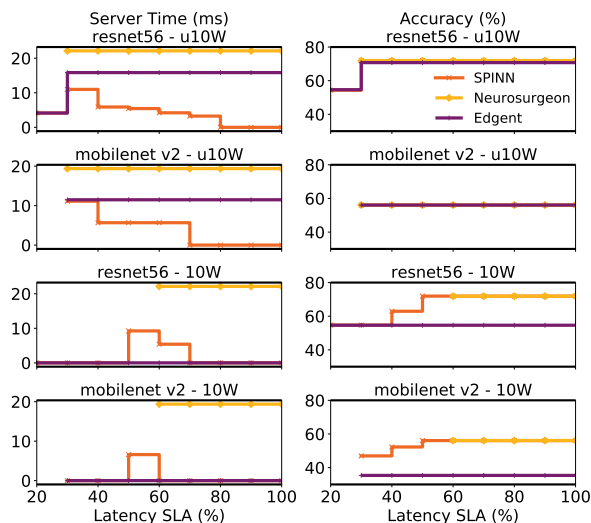


图 9: 针对不同延迟 SLA 和客户端计算能力 (u10W 和 10W), SPINN 与 Neurosurgeon 和 Edgent 的服务器时间 (左) 和准确性 (右)。SLA 以设备上延迟的百分比表示。

4.2.2 服务器负载变化。为了研究 SPINN 在各种服务器端负载下的性能, 我们在改变远程端负载时相对于基线测量了 SPINN 的推理吞吐量, 精度下降容限为 1pp。这是通过按减速因子线性缩放服务器执行的延迟来实现的 (即因子 2 意味着服务器慢 2x)。图 8 显示了每种方法在各种服务器端负载下实现的吞吐量, 其中 Jetson 配置为 10W 配置文件, 网络速度为 WiFi-5 范围 (500 Mbps)。

在服务器负载较低 (x 轴左侧) 的情况下, 所检查的系统表现出与图 7 的高带宽性能类似的趋势。随着服务器负载增加 (即朝向右侧), 性能会恶化, 但仅设备执行的情况除外, 该情况对于服务器负载是不变的。一方面, 尽管其可达到的吞吐量有所下降, 但 Neurosurgeon 会根据服务器利用率调整其策略, 并逐渐在客户端执行更大部分的 CNN。另一方面, Edgent 的吞吐量下降得更快, 甚至在高服务器负载下低于仅设备执行, 因为它的运行时机制没有考虑变化的服务器负载。相反, 通过自适应优化分割点和提前退出策略, SPINN 的调度程序能够根据服务器端负载调整整体执行, 从而实现 1.18-1.99x (1.57x geo 之间的吞吐量增益。均值) 和 1.15-3.09x (1.61x 地理位置。分别对 Neurosurgeon 和 Edgent 进行平均)。

4.2.3 案例研究: 以最低服务器成本实现延迟驱动的 SLA。为了评估 SPINN 在截止日期内的性能, 我们针对这样的场景: 服务提供商旨在部署基于 CNN 的应用程序, 以最大的准确性和最低的服务器端成本满足严格的延迟 SLA。在此设置中, 我们与 Neurosurgeon8 和 Edgent 进行比较, 针对 4G 上的 MobileNetV2 和 ResNet-56。图 9 显示了每个服务器的计算时间和准确率

适用于具有不同计算能力的两个设备配置文件的系统 - u10W 和 10W。延迟 SLA 以原始 CNN 纯设备运行时的百分比表示 (即 20% SLA 意味着目标延迟比设备上执行少 5x, 需要提前退出和/或服务器支持)。

对于低端设备 (u10W) 和严格的延迟期限, SPINN 尽可能将负载卸载到云端, 因为它可以更快地到达网络中的后续出口, 从而提高准确性。随着 SLA 放松 (达到设备上延迟的 40% 以上), SPINN 开始逐渐在本地执行越来越多的操作。相比之下, Edgent 和 Neurosurgeon 的准确度相似, 但服务器负载分别高出 4.9x 和 6.8x。平均而言, 由于其灵活的多目标调度程序, SPINN 将 Edgent 和 Neurosurgeon 服务器时间分别减少了 68.64% 和 82.5% (60.3% 和 83.6% geo.mean)。相反, 神经外科医生只能针对整体延迟进行优化, 而不能牺牲准确性来满足最后期限 (例如, ResNet-56 上的 20% SLA), 而 Edgent 无法考虑服务器时间最小化和准确性下降约束。

对于更强大的设备 (10W), 情况有所不同。随着设备速度越来越快, SLA 目标变得更加严格。因此, 我们观察到 SPINN 和 Edgent 仍然可以分别满足 ResNet-56 和 MobileNetV2 本地执行时间低至 20% 和 30% 的延迟约束。相比之下, 如果没有渐进式推理, 神经外科医生不可能在两个 CNN 上实现设备上执行的推理延迟低于 60%。在这种情况下, SPINN 能够权衡准确性以满足更严格的 SLA, 但随着延迟限制的放宽, 也可以提高其可达到的准确性。

对于较宽松的延迟期限 (目标大于设备上延迟的 50%), 对于 ResNet-56 和 MobileNetV2, SPINN 的准确率分别比 Edgent 提高了 17.3% 和 20.7%。这背后的原因是双重的。首先, 在卸载时, Edgent 在服务器端启动计算, 增加了通信延迟开销。相反, SPINN 的客户端到服务器卸载策略和压缩显著降低了通信延迟开销。其次, 由于 Edgent 的非标准化成本函数 (即 $\max(1 \text{ lat} + \text{acc})$), 吞吐量的奖励主导了准确性增益, 导致总是选择第二个提前退出子网络并在本地执行。相比之下, SPINN 调度程序的多标准设计能够捕获准确性、服务器时间和延迟约束, 以产生优化的部署。因此, 与速度较慢的设备类似, SPINN 成功地利用服务器资源在延迟限制下提高准确性, 同时它可以在宽松的期限内达到纯粹的设备上执行。

4.3 运行时开销和效率

4.3.1 部署开销。通过对我们检查的 CNN 和 Jetson CPU 上的数据集进行评估, 调度程序的执行时间最长为 14 毫秒 (geo.mean 为 11 毫秒)。该时间包括读取分析器参数、更新监控指标以及搜索和返回所选配置的成本。此外, SPINN 的内存消耗约为几 KB (即 Jetson RAM 的 <1%)。这些成本通过多次推理进行摊销, 因为调度程序仅在重大上下文更改时被调用。我们将在下一节中讨论此类参数的选择。

⁸It should be noted that Neurosurgeon maintains the accuracy of the original CNN.

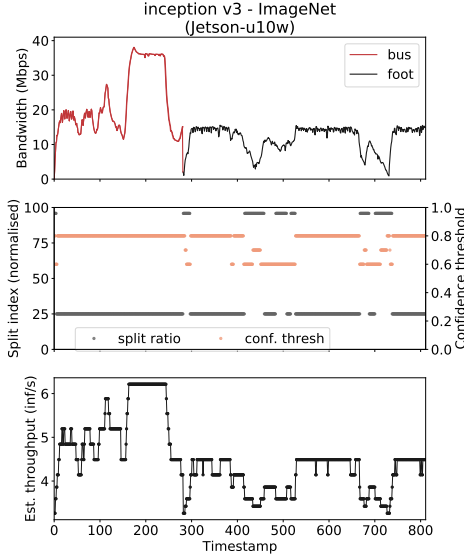


Figure 10: SPINN scheduler’s behaviour on real network provider trace.

4.3.2 Network Variation. To assess the responsiveness of SPINN in adapting to dynamic network conditions, we targeted a real bandwidth trace from a Belgian ISP. The trace contains time series of network bandwidth variability during different user activities. In this setup, SPINN executes the ImageNet-trained Inception-v3 with Jetson-u10W as the client under the varying bandwidth emulated by the Belgium 4G/LTE logs. The scheduler is configured to maximise both throughput and accuracy. Figure 10 (top) shows an example bandwidth trace from a moving bus followed by walking. Figure 10 (bottom) shows SPINN’s achieved inference throughput under the changing network quality. The associated scheduler decisions are depicted in Figure 10 (middle).

At low bandwidth (<5 Mbps), SPINN falls back to device-only execution. In these cases, the scheduler adopts a less conservative early-exit policy by lowering the confidence threshold. In this manner, it allows more samples to exit earlier, compensating for the client’s low processing power. Nonetheless, the impact on accuracy remains minimal (<1%) for the selected early-exit policies by the scheduler ($thr_{conf} \in [0.6, 1.0]$), as illustrated in Figure 3 for Inception-v3 on ImageNet. At the other end, high bandwidths result in selecting an earlier split point and thus achieving up to 7× more inf/sec over pure on-device execution. Finally, the similar trajectories of the top and bottom figure suggest that our scheduler can adapt the system to the running conditions, without having to be continuously invoked.

Overall, we observe that small bandwidth changes do not cause significant alterations to the split and early-exit strategies. By employing an averaging historical window of three values and a difference threshold of 5%, the scheduler is invoked 1/3 of the total bandwidth changes across traces.

4.4 Energy Consumption

Figure 11 shows the breakdown of dominant energy consumption across the client device subsystems. We measured energy consumption over 1000 inferences from the validation set and offloading

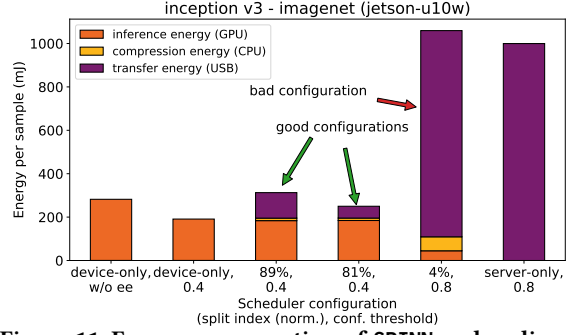


Figure 11: Energy consumption of SPINN vs. baselines.

over UK’s Three Broadband’s 4G network with a Huawei E3372 USB adapter. We measured the power of Jetson (CPU, GPU) from its integrated probes and the transmission energy with the Monsoon AAA10F power monitor.

Traversing the horizontal axis left-to-right, we first see device-only execution without and with early-exits, where the local processing dominates the total energy consumption. The latter shows benefits due to samples exiting early from the network. Next, we showcase the consumption breakdown of three different $\langle split, thr_{conf} \rangle$ configurations. The first two configurations demonstrate comparable energy consumption with the device-only execution without early exits. On the contrary, a bad configuration requires an excessively large transfer size, leading to large compression and transfer energy overheads. Last, the energy consumption when fully offloading is dominated by the network transfer.

Across configurations, we witness a 5× difference in energy consumption across different inference setups. While device-only execution yields the lowest energy footprint per sample, it is also the slowest. Our scheduler is able to yield deployments that are significantly more energy efficient than full offloading (4.2×) and on par with on-device execution (0.76 – 1.12×), while delivering significantly faster end-to-end processing. Finally, with different configurations varying both in energy and performance, the decision space is amenable to energy-driven optimisation by adding energy as a scheduler optimisation metric.

4.5 Constrained Availability Robustness

Next we evaluate SPINN’s robustness under constrained availability of the remote end such as network timeouts, disconnections and server failures. More specifically, we investigate 1) the achieved accuracy across various failure rates and 2) the latency improvement over conventional systems enhanced with an error-control policy. In these experiments, we fix the confidence threshold of three models (Inception-v3, ResNet-56 and ResNet-50) to a value of 0.8 and emulate variable failure rates by sampling from a random distribution across the validation set.

Accuracy comparison: For this experiment (Figure 12), we compare SPINN at different network split points (solid colours) against a non-progressive baseline (dashed line). Under failure conditions, the baseline unavoidably misclassifies the result as there is no usable result locally on-device. However, SPINN makes it possible to acquire the most confident local result up to the split point, when the server is unavailable.

As shown in Figure 12, the baseline quickly drops in accuracy as the failure rate increases. This is not the case with SPINN, which

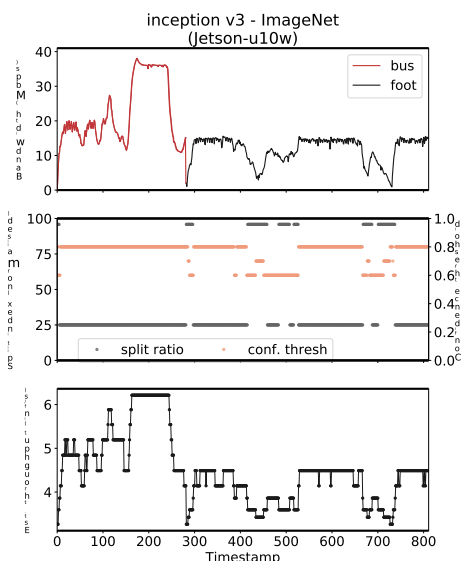


图 10: SPINN 调度程序在真实网络提供商跟踪上的行为。

4.3.2 网络变化。为了评估 SPINN 在适应动态网络条件方面的响应能力，我们针对来自比利时 ISP 的真实带宽跟踪。该跟踪包含不同用户活动期间网络带宽变化的时间序列。在此设置中，SPINN 在比利时 4G/LTE 日志模拟的不同带宽下以 Jetson-u10w 作为客户端执行 ImageNet 训练的 Inception-v3。调度程序被配置为最大化吞吐量和准确性。图 10（上）显示了移动公交车随后步行的带宽跟踪示例。图 10（下）显示了 SPINN 在不断变化的网络质量下实现的推理吞吐量。相关的调度程序决策如图 10（中）所示。

在低带宽 (<5 Mbps) 下，SPINN 回退到仅设备执行。在这些情况下，调度程序通过降低置信度阈值来采用不太保守的提前退出策略。通过这种方式，它允许更多样本更早退出，从而补偿客户端的低处理能力。尽管如此，调度程序 ($\text{thrconf} \in [0.6, 1.0]$) 选择的提前退出策略对准确性的影响仍然很小 (<1%)，如图 3 中 ImageNet 上的 Inception-v3 所示。另一方面，高带宽导致选择较早的分割点，从而比纯设备上执行多实现 $7 \times \text{inf/秒}$ 。最后，上图和下图的相似轨迹表明我们的调度程序可以使系统适应运行条件，而无需不断调用。

总体而言，我们观察到较小的带宽变化不会导致拆分和提前退出策略发生重大变化。通过采用三个值的平均历史窗口和 5% 的差异阈值，调度程序将被调用，跨跟踪的总带宽变化为 $1/3$ 。

4.4 能源消耗

图 11 显示了客户端设备子系统的主要能耗细分。我们根据验证集和卸载测量了 1000 多个推论的能耗

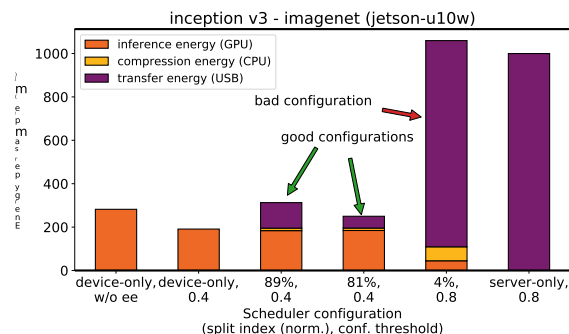


图 11: SPINN 的能耗与基线。

使用华为 E3372 USB 适配器通过英国三宽带的 4G 网络。我们通过集成探针测量了 Jetson (CPU、GPU) 的功率，并使用 Monsoon AAA10F 功率监视器测量了传输能量。

从左到右穿过水平轴，我们首先看到没有或有提前退出的仅设备执行，其中本地处理主导了总能耗。后者显示出由于样本提前从网络中退出而带来的好处。接下来，我们展示三种不同的 $\langle \text{split}, \text{thrconf} \rangle$ 的消耗明细

配置。前两种配置展示了与仅设备执行（无需提前退出）相当的能耗。相反，糟糕的配置需要过大的传输大小，从而导致较大的压缩和传输能量开销。最后，完全卸载时的能耗主要由网络传输决定。

在各种配置中，我们发现不同推理设置的能耗存在 $5 \times$ 差异。虽然仅设备执行每个样本的能耗最低，但它也是最慢的。我们的调度程序能够产生比完全卸载 ($4.2 \times$) 更加节能的部署，并且与设备上执行 ($0.76 - 1.12 \times$) 相当，同时提供更快的端到端处理。最后，由于不同的配置在能量和性能方面都有所不同，因此决策空间可以通过添加能量作为调度程序优化指标来进行能量驱动优化。

4.5 受限可用性鲁棒性

接下来，我们评估 SPINN 在远程端可用性受限（例如网络超时、断开连接和服务端故障）情况下的鲁棒性。更具体地说，我们研究了 1) 在不同故障率下所实现的准确性，以及 2) 通过错误控制策略增强的传统系统的延迟改进。在这些实验中，我们将三个模型（Inception-v3、ResNet-56 和 ResNet-50）的置信阈值固定为 0.8，并通过从验证集的随机分布中采样来模拟可变失败率。

准确性比较：对于本实验（图 12），我们将不同网络分割点（实心颜色）的 SPINN 与非渐进基线（虚线）进行比较。在故障情况下，基线不可避免地会对结果进行错误分类，因为设备本地没有可用的结果。然而，当服务器不可用时，SPINN 可以获取分割点之前最可信的本地结果。

如图 12 所示，随着故障率的增加，基线的准确度迅速下降。SPINN 的情况并非如此，

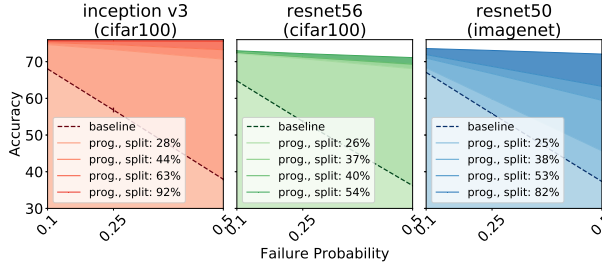


Figure 12: Comparison of the average accuracy under uncertain server availability. The shaded area indicates attained accuracies under a probability distribution.

manages to maintain a minimal accuracy drop. Specifically, we witness drops ranging in $[0, 5.75\%]$ for CIFAR-100 and $[0.46\%, 33\%]$ for ImageNet, when the equivalent drop of the baseline is $[11.56\%, 44.1\%]$ and $[9.25\%, 44.34\%]$, respectively. As expected, faster devices are able to execute locally a larger part of the model (exit later) while meeting their SLA exhibit the smallest impact under failure, as depicted in the progressive variants of the two models.

Latency comparison: In this scenario, we compare SPINN against a single-exit offloaded variant of the same networks. This time instead of simply failing the inference when the remote end is unavailable, we allow for *retransmission* with *exponential back-off*, a common behaviour of distributed systems to avoid channel contention. When a sample fails under the respective probability distribution, the result gets retransmitted. If the same sample fails again, the client waits double the time and retransmits, until it succeeds. Here, we assume Jetson-10W offloads to our server over 4G and varying failure probability ($P_{\text{fail}} \in \{0.1, 0.25, 0.5\}$). The initial retransmission latency is 20 ms. We ran each experiment three times and report the mean and standard deviation of the latencies.

As depicted in Figure 13, the non-progressive baseline follows a trajectory of increasing latency as the failure probability gets higher, due to the additional back-off latency each time a sample fails. While the impact on the average latency for both networks going from $P_{\text{fail}} = 0.1$ to $P_{\text{fail}} = 0.25$ is gradual, at 3.9%, 5.8% and 4.7% for Inception-v3, ResNet-56 and ResNet-50 respectively, the jump from $P_{\text{fail}} = 0.25$ to $P_{\text{fail}} = 0.5$ is much more dramatic, at 52.9%, 91% and 118%. The variance at $P_{\text{fail}} = 0.5$ is also noticeably higher, compared to previous values, attributed to higher number of retransmissions and thus higher discrepancies across different runs. We should note that despite the considerably higher latency of the non-progressive baseline, its accuracy can be higher, since all samples – whether re-transmitted or not – exit at the final classifier. Last, we also notice a slight reduction in the average latency of SPINN’s models as P_{fail} increases. This is a result of more samples early-exiting in the network, as the server becomes unavailable more often.

To sum up, these two results demonstrate that SPINN can perform sufficiently, in terms of accuracy and latency, even when the remote end remains unresponsive, by falling back to results of local exits. Compared to other systems, as the probability of failure when offloading to the server increases, there is a gradual degradation of the quality of service, instead of catastrophic unresponsiveness of the application.

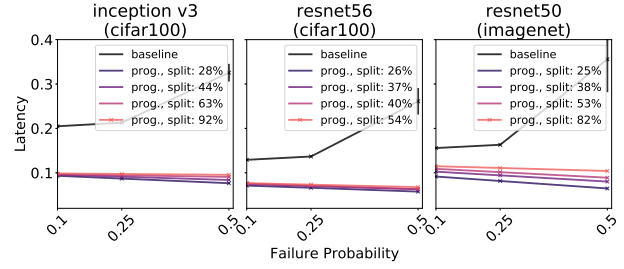


Figure 13: Comparison of average latency under uncertain server availability.

5 DISCUSSION

SPINN and the current ML landscape. The status-quo deployment process of CNNs encompasses the maintenance of two models: a large, highly accurate model on the cloud and a compact, lower-accuracy one on the device. However, this approach comes with significant deployment overheads. First, from a development time perspective, the two-model approach results in two time- and resource-expensive stages. In the first stage, the large model is designed and trained requiring multiple GPU-hours. In the second stage, the large model is compressed through various techniques in order to obtain its lightweight counterpart, with the selection and tuning of the compression method being a difficult task in itself. Furthermore, typically, to gain back the accuracy loss due to compression, the lightweight model has to be fine-tuned through a number of additional training steps.

With regards to the lightweight compressed networks, SPINN is orthogonal to these techniques and hence a compressed model can be combined with SPINN to obtain further gains. Given a compressed model, our system would proceed to derive a progressive inference variant with early exits and deploy the network with a tailored implementation. For use-cases where pre-trained models are employed, SPINN can also smoothly be adopted by modifying its training scheme (Section 3.2) so that the pre-trained backbone is frozen during training and only the early exits are updated.

Nonetheless, with SPINN we also enable an alternative paradigm that alleviates the main limitations of the current practice. SPINN requires a single network design step and a single training process - which trains both the backbone network and its early exits. Upon deployment, the model’s execution is adaptively tuned based on the multiple target objectives, the environmental conditions and the device and cloud load. In this manner, SPINN enables a highly customised deployment which is dynamically and efficiently adjusted to sustain its performance in mobile settings. This approach is further supported by the ML community’s growing number of works on progressive networks [23, 35, 48, 72, 81, 83, 84] which can be directly targeted by SPINN to yield an optimised deployment on mobile platforms.

Limitations and future work. Despite the challenges addressed by SPINN, our prototype system has certain limitations. First, the scheduler does not explicitly optimise for energy or memory consumption of the client. The energy consumption could be integrated as another objective in the MOO solver of the scheduler, while memory footprint could be minimised by only loading part of the model in memory and always offloading the rest. Moreover, while

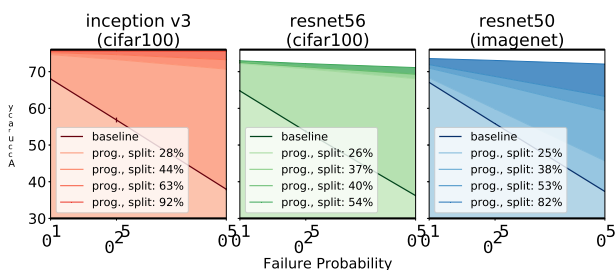


图 12: 服务器可用性不确定情况下的平均准确度比较。阴影区域表示在概率分布下获得的精度。

设法保持最小的精度下降。具体来说，当基线的等效下降分别为 [11.56%、44.1%] 和 [9.25%、44.34%] 时，CIFAR-100 的下降范围为 [0、5.75%]，ImageNet 的下降范围为 [0.46%、33%]。正如预期的那样，速度更快的设备能够在本地执行模型的大部分（稍后退出），同时满足 SLA，在出现故障时表现出最小的影响，如两个模型的渐进变体所示。

延迟比较：在这种情况下，我们将 SPINN 与相同网络的单出口卸载变体进行比较。这次，我们不是在远程端不可用时简单地使推理失败，而是允许使用指数退避重新传输，这是分布式系统避免通道争用的常见行为。当样本在相应的概率分布下失败时，结果将被重新传输。如果同一样本再次失败，客户端将等待双倍时间并重新传输，直到成功。在这里，我们假设 Jetson-10W 通过 4G 和不同的故障概率卸载到我们的服务器 ($P_{fail} \in \{0.1, 0.25, 0.5\}$)。初始重传延迟为 20 ms。我们将每个实验运行三次并报告延迟的平均值和标准偏差。

如图 13 所示，随着失败概率变高，非渐进基线遵循延迟增加的轨迹，这是由于每次样本失败时都会产生额外的回退延迟。虽然从 $P_{fail} = 0.1$ 到 $P_{fail} = 0.25$ 对两个网络的平均延迟的影响是渐进的，对于 Inception-v3、ResNet-56 和 ResNet-50 分别为 3.9%、5.8% 和 4.7%，但从 $P_{fail} = 0.25$ 到 $P_{fail} = 0.5$ 的跳跃要戏剧性得多，在 52.9%、91% 和 118%。与之前的值相比， $P_{fail} = 0.5$ 处的方差也明显较高，这归因于重传次数较多，因此不同运行之间的差异较大。我们应该注意到，尽管非渐进基线的延迟相当高，但其准确性可能更高，因为所有样本（无论是否重新传输）都会在最终分类器处退出。最后，我们还注意到，随着 P_{fail} 的增加，SPINN 模型的平均延迟略有减少。这是由于更多样本提前退出网络的结果，因为服务器更频繁地变得不可用。

综上所述，这两个结果表明，即使在远程端保持无响应的情况下，SPINN 也可以通过回退到本地退出的结果，在准确性和延迟方面表现良好。与其他系统相比，随着卸载到服务器时失败的可能性增加，服务质量会逐渐下降，而不是应用程序出现灾难性的无响应。

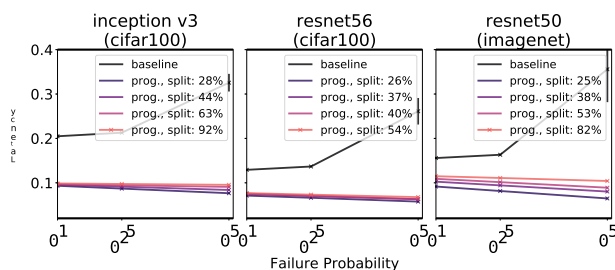


图 13: 服务器可用性不确定情况下的平均延迟比较。

5 讨论

SPINN 和当前的 ML 格局。CNN 的现状部署过程包括维护两种模型：云上的大型、高精度模型和设备上的紧凑、精度较低的模型。然而，这种方法会带来巨大的部署开销。首先，从开发时间的角度来看，两种模型方法导致两个耗时和资源昂贵的阶段。在第一阶段，大型模型的设计和训练需要多个 GPU 小时。在第二阶段，通过各种技术对大型模型进行压缩以获得轻量级模型，压缩方法的选择和调整本身就是一项艰巨的任务。此外，通常，为了弥补由于压缩而导致的准确性损失，必须通过许多额外的训练步骤对轻量级模型进行微调。

对于轻量级压缩网络，SPINN 与这些技术正交，因此压缩模型可以与 SPINN 相结合以获得进一步的增益。给定一个压缩模型，我们的系统将继续导出具有早期退出的渐进推理变体，并通过定制的实现来部署网络。对于使用预训练模型的用例，SPINN 也可以通过修改其训练方案（第 3.2 节）顺利采用，以便在训练期间冻结预训练的主干，仅更新早期的退出。

尽管如此，通过 SPINN，我们还实现了一种替代范式，减轻了当前实践的主要局限性。SPINN 需要单一的网络设计步骤和单一的训练过程——训练主干网络及其早期出口。部署后，模型的执行会根据多个目标、环境条件以及设备和云负载进行自适应调整。通过这种方式，SPINN 可以实现高度定制的部署，并进行动态、高效的调整，以维持其在移动环境中的性能。这种方法得到了 ML 社区越来越多的渐进式网络研究的进一步支持 [23,35,48,72,81,83,84]，SPINN 可以直接瞄准这些网络，以在移动平台上产生优化的部署。

局限性和未来的工作。尽管 SPINN 解决了这些挑战，但我们的原型系统仍存在一定的局限性。首先，调度程序没有明确优化客户端的能量或内存消耗。能源消耗可以作为另一个目标集成到调度程序的 MOO 求解器中，而通过仅将模型的一部分加载到内存中并始终卸载其余部分，可以最大限度地减少内存占用。此外，虽然

SPINN supports splitting at any given layer, we limit the candidate split points of each network to the outputs of ReLU layers, due to their high compressibility (Section 3.3). Although offloading could happen at sub-layer, filter-level granularity, this would impose extra overhead on the scheduler due to the significantly larger search space.

Our workflow also assumes the model to be available at both the client and server side. While cloud resources are often dedicated to specific applications, edge resources tend to present locality challenges. To handle these, we could extend SPINN to provide incremental offloading [29] and cache popular functionality [76] closer to its users. In the future, we intend to explore multi-client settings and simultaneous asynchronous inferences on a single memory copy of the model, as well as targeting regression tasks and recurrent neural networks.

6 CONCLUSION

In this paper, we present SPINN, a distributed progressive inference engine that addresses the challenge of partitioning CNN inference across device-server setups. Through a run-time scheduler that jointly tunes the early-exit policy and the partitioning scheme, the proposed system supports complex performance goals in highly dynamic environments while simultaneously guaranteeing the robust operation of the end system. By employing an efficient multi-objective optimisation approach and a CNN-specific communication optimiser, SPINN is able to deliver higher performance over the state-of-the-art systems across diverse settings, without sacrificing the overall system’s accuracy and availability.

REFERENCES

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI)*. 265–283.
- [2] Mario Almeida, Stefanos Laskaridis, Ilias Leontiadis, Stylianos I. Venieris, and Nicholas D. Lane. 2019. EmBench: Quantifying Performance Variations of Deep Neural Networks Across Modern Commodity Devices. In *The 3rd International Workshop on Deep Learning for Mobile Systems and Applications (EMDL)* (Seoul, Republic of Korea). 1–6.
- [3] Amazon. 2020. Amazon Inferentia ML Chip. <https://aws.amazon.com/machine-learning/inferentia/>. [Retrieved: August 25, 2020].
- [4] Alejandro Cartas, Martín Kocour, Aravindh Raman, Ilias Leontiadis, Jordi Luque, Nishanth Sastry, Jose Nuñez Martinez, Diego Perino, and Carlos Segura. 2019. A Reality Check on Inference at Mobile Networks Edge. In *Proceedings of the 2nd International Workshop on Edge Systems, Analytics and Networking (EdgeSys)*. 54–59.
- [5] D. Chandrasekar. 2016. *AWS Flap Detector: An Efficient Way to Detect Flapping Auto Scaling Groups on AWS Cloud*. University of Cincinnati.
- [6] E. Chung et al. 2018. Serving DNNs in Real Time at Datacenter Scale with Project Brainwave. *IEEE Micro* 38, 2 (2018), 8–20.
- [7] E. Chung, J. Fowers, K. Ovtcharov, M. Papamichael, A. Caulfield, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, M. Abeydeera, L. Adams, H. Angepat, C. Boehn, D. Chiou, O. Firestein, A. Forin, K. S. Gatlin, M. Ghandi, S. Heil, K. Holohan, A. El Hussein, T. Juhasz, K. Kagi, R. Kovvuri, S. Lanka, F. van Megen, D. Mukhortov, P. Patel, B. Perez, A. Rapsang, S. Reinhardt, B. Rouhani, A. Sapek, R. Seera, S. Shekar, B. Sridharan, G. Weisz, L. Woods, P. Yi Xiao, D. Zhang, R. Zhao, and D. Burger. 2018. Serving DNNs in Real Time at Datacenter Scale with Project Brainwave. *IEEE Micro* 38, 2 (2018), 8–20.
- [8] A. E. Eshratifar, M. S. Abrishami, and M. Pedram. 2019. JointDNN: An Efficient Training and Inference Engine for Intelligent Mobile Cloud Computing Services. *IEEE Transactions on Mobile Computing (TMC)* (2019).
- [9] Biyi Fang, Xiao Zeng, and Mi Zhang. 2018. NestDNN: Resource-Aware Multi-Tenant On-Device Deep Learning for Continuous Mobile Vision. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking (MobiCom)*. 115A–127.
- [10] L. Fei-Fei, J. Deng, and K. Li. 2010. ImageNet: Constructing a large-scale image database. *Journal of Vision* 9, 8 (2010), 1037–1037.
- [11] L. Fridman, D. E. Brown, M. Glazer, W. Angell, S. Dodd, B. Jenik, J. Terwilliger, A. Patsek, J. Kindelsberger, L. Ding, S. Seaman, A. Mehler, A. Sipperley, A. Pettinato, B. D. Seppelt, L. Angell, B. Mehler, and B. Reimer. 2019. MIT Advanced Vehicle Technology Study: Large-Scale Naturalistic Driving Study of Driver Behavior and Interaction With Automation. *IEEE Access* 7 (2019), 102021–102038.
- [12] Evangelos Georganas, Sasikanth Avancha, Kunal Banerjee, Dhiraj Kalamkar, Greg Henry, Hans Pabst, and Alexander Heinecke. 2018. Anatomy of High-Performance Deep Learning Convolutions on SIMD Architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC)* (SC ’18). IEEE Press, Article 66, 12 pages.
- [13] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*. 1321–1330.
- [14] Kaiyuan Guo, Lingzhi Sui, Jiantao Qiu, Jincheng Yu, Junbin Wang, Song Yao, Song Han, Yu Wang, and Huazhong Yang. 2017. Angel-Eye: A complete design flow for mapping CNN onto embedded FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 37, 1 (2017), 35–47.
- [15] Selim Gurun, Chandra Krintz, and Rich Wolski. 2004. NWSLite: A Light-Weight Prediction Utility for Mobile Devices. In *Proceedings of the 2nd International Conference on Mobile Systems, Applications, and Services (MobiSys)*. 2A–11.
- [16] P. Gysel, J. Pimentel, M. Motamedi, and S. Ghiasi. 2018. Ristretto: A Framework for Empirical Study of Resource-Efficient Inference in Convolutional Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)* 29, 11 (2018), 5784–5789.
- [17] Song Han, Huizi Mao, and William J Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *International Conference on Learning Representations (ICLR)* (2016).
- [18] Seungyeop Han, Haichen Shen, Matthai Philipose, Sharad Agarwal, Alec Wolman, and Arvind Krishnamurthy. 2016. MCDNN: An Approximation-Based Execution Framework for Deep Stream Processing Under Resource Constraints. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*. 123–136.
- [19] K. Hazelwood, S. Bird, D. Brooks, S. Chintala, U. Diril, D. Dzhulgakov, M. Fawzy, B. Jia, Y. Jia, A. Kalro, J. Law, K. Lee, J. Lu, P. Noordhuis, M. Smelyanskiy, L. Xiong, and X. Wang. 2018. Applied Machine Learning at Facebook: A Datacenter Infrastructure Perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 620–629.
- [20] K He, X Zhang, S Ren, and J Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778.
- [21] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodik, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B. Gibbons, and Onur Mutlu. 2018. Focus: Querying Large Video Datasets with Low Latency and Low Cost. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 269–286.
- [22] C. Hu, W. Bao, D. Wang, and F. Liu. 2019. Dynamic Adaptive DNN Surgery for Inference Acceleration on the Edge. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*. 1423–1431.
- [23] Gao Huang, Danlu Chen, Tianhong Li, Felix Wu, Laurens van der Maaten, and Kilian Weinberger. 2018. Multi-Scale Dense Networks for Resource Efficient Image Classification. In *International Conference on Learning Representations (ICLR)*.
- [24] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2017. Quantized Neural Networks: Training Neural Networks with Low Precision Weights and Activations. *J. Mach. Learn. Res.* 18, 1 (2017), 6869–6898.
- [25] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. 2019. AI Benchmark: All About Deep Learning on Smartphones in 2019. In *International Conference on Computer Vision (ICCV) Workshops*.
- [26] UK ISPs. 2020. 4G Mobile Network Experience Report. <https://www.opensignal.com/reports/2019/04/uk/mobile-network-experience>.
- [27] UK ISPs. 2020. 5G Mobile Network Report. <https://www.opensignal.com/2020/02/20/how-att-sprint-t-mobile-and-verizon-differ-in-their-early-5g-approach>.
- [28] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.
- [29] Hyuk-Jin Jeong, Hyeon-Jae Lee, Chang Hyun Shin, and Soo-Mook Moon. 2018. IONN: Incremental Offloading of Neural Network Computations from Mobile Devices to Edge Servers. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*. 401–411.
- [30] Yu Ji, Youhui Zhang, Wenguang Chen, and Yuan Xie. 2018. Bridge the Gap Between Neural Networks and Neuromorphic Hardware with a Neural Network

SPINN 支持在任何给定层进行分割, 由于 ReLU 层的高压缩性, 我们将每个网络的候选分割点限制为 ReLU 层的输出 (第 3.3 节)。尽管卸载可能发生在子层、过滤器级别的粒度上, 但由于搜索空间明显更大, 这会给调度程序带来额外的开销。

我们的工作流程还假设该模型在客户端和服务端均可用。虽然云资源通常专用于特定应用程序, 但边缘资源往往会带来局部性挑战。为了处理这些问题, 我们可以扩展 SPINN 以提供增量卸载 [29] 并缓存更靠近用户的流行功能 [76]。未来, 我们打算探索多客户端设置和对模型的单个内存副本进行同步异步推理, 以及针对回归任务和循环神经网络。

6 结论

在本文中, 我们提出了 SPINN, 这是一种分布式渐进推理引擎, 可解决跨设备-服务器设置划分 CNN 推理的挑战。通过联合调整提前退出策略和分区方案的运行时调度程序, 所提出的系统支持高度动态环境中的复杂性能目标, 同时保证终端系统的稳健运行。通过采用高效的多目标优化方法和 CNN 特定的通信优化器, SPINN 能够在不同的设置中提供比最先进的系统更高的性能, 而不会牺牲整个系统的准确性和可用性。

参考

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, 于媛, 郑小强. 2016. TensorFlow: 大规模机器学习系统。第 12 届 USENIX 操作系统设计与实现 (OSDI) 会议论文集。265–283。[2] Mario Almeida, Stefanos Laskaridis, Ilias Leontiadis, Stylianos I. Venieris 和 Nicholas D. Lane. 2019. EmBench: 量化现代商品设备中深度神经网络的性能变化。第三届移动系统和应用深度学习 (EMDL) 国际研讨会 (韩国首尔)。1–6。[3] 亚马逊。2020. 亚马逊 Inferentia 机器学习芯片。https://aws.amazon.com/machine-learning/inferentia/。[检索日期: 2020 年 8 月 25 日]。[4] 亚历杭德罗·卡塔斯、马丁·科库尔、阿拉文德·拉曼、伊利亚斯·莱昂蒂亚迪斯、乔迪·卢克、尼尚斯·萨斯特里、何塞·努涅斯·马丁内斯、迭戈·佩里诺和卡洛斯·塞古拉。2019. 移动网络边缘推理的现实检验。第二届边缘系统、分析和网络国际研讨会 (EdgeSys) 论文集。54–59。[5] D. 钱德拉塞卡。2016. AWS Flap Detector: 一种在 AWS 云上检测浮动 Auto Scaling 组的有效方法。辛辛那提大学。[6] E. Chung 等人。2018 年。通过 Project Brainwave 在数据中心规模实时提供 DNN 服务。IEEE Micro 38, 2 (2018), 8–20。[7] E. Chung, J. Fowers, K. Ovtcharov, M. Papamichael, A. Caulfield, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, M. Abeydeera, L. Adams, H. Angepat, C. Boehn, D. Chiou, O. Firestein, A. Forin, K. S. Gatlin, M. Ghandi, S. Heil, K. Holohan, A. El Hussein, T. Juhasz, K. Kagi, R. Kovvuri, S. Lanka, F. van Megen, D. Mukhortov, P. Patel, B. Perez, A. Rapsang, S. Reinhardt, B. Rouhani, A. Sapek, R. Seera, S. Shekar, B. Sridharan, G. Weisz, L. Woods, P. 肖毅, 张 D. 赵, D. Burger。2018 年。通过 Project Brainwave 在数据中心规模实时提供 DNN 服务。IEEE Micro 38, 2 (2018), 8–20。[8] A. E. Eshratifar, M. S. Abrishami 和 M. Pedram。2019. JointDNN: 智能移动云计算服务的高效训练和推理引擎。IEEE 移动计算汇刊 (TMC) (2019)。[9] 方必一, 曾晓, 张宓. 2018. NestDNN: 用于连续移动视觉的资源感知多租户设备上深度学习。诉讼中

第 24 届年度移动计算和网络国际会议 (MobiCom) 的会议。115 至 127。[10] L. 飞飞, J. 邓, K. 李。2010. ImageNet: 构建大规模图像数据库。视觉杂志 9, 8 (2010), 1037–1037。[11] L. Fridman, D. E. Brown, M. Glazer, W. Angell, S. Dodd, B. Jenik, J. Terwilliger, A. Patsek, J. Kindelsberger, L. Ding, S. Seaman, A. Mehler, A. Sipperley, A. Pettinato, B. D. Seppelt, L. Angell, B. Mehler 和 B. Reimer。2019. 麻省理工学院先进车辆技术研究: 驾驶员行为和与自动化交互的大规模自动驾驶研究。IEEE Access 7 (2019), 102021–102038。[12] Evangelos Georganas, Sasikanth Avancha, Kunal Banerjee, Dhiraj Kalamkar, Greg Henry, Hans Pabst 和 Alexander Heinecke。2018. SIMD 架构上的高性能深度学习卷积剖析。高性能计算、网络、存储和分析 (SC) 国际会议记录 (SC 会议)。IEEE Press, 第 66 条, 12 页。[13] 郭川, Geoff Pleiss, 孙宇, Kilian Q. Weinberger。2017. 关于现代神经网络的校准。第 34 届国际机器学习会议 (ICML) 论文集。1321–1330。[14] 郭开元, 隋凌志, 邱建涛, 于金城, 王俊斌, 姚松, 韩松, 王宇, 杨华中。2017. Angel-Eye: 将 CNN 映射到嵌入式 FPGA 的完整设计流程。IEEE 集成电路和系统计算机辅助设计 (TCAD) 交易 37, 1 (2017), 35–47。[15] 塞利姆·古伦、钱德拉·克林茨和里奇·沃尔斯基。2004. NWSLite: 移动设备的轻量级预测实用程序。第二届国际移动系统、应用程序和服务会议 (MobiSys) 论文集。2、11。[16] P. Gysel, J. Pimentel, M. Motamedi 和 S. Ghiasi。2018. Ristretto: 卷积神经网络中资源高效推理的实证研究框架。IEEE 神经网络和学习系统汇刊 (TNNLS) 29, 11 (2018), 5784–5789。[17] 韩松, 毛慧子, William J. Dally。2016. 深度压缩: 通过剪枝、训练量化和霍夫曼编码来压缩深度神经网络。国际学习表征会议 (ICLR) (2016)。[18] Seungyeop Han, Haicheng Shen, Matthai Philipose, Sharad Agarwal, Alec Wolman 和 Arvind Krishnamurthy。2016. MCDNN: 资源约束下基于近似的深度流处理执行框架。第 14 届移动系统、应用程序和服务国际会议 (MobiSys) 的会议记录。123–136。[19] K. Hazelwood, S. Bird, D. Brooks, S. Chintala, U. Diril, D. Dzulgakov, M. Fawzy, B. Jia, Y. Jia, A. Kalro, J. Law, K. Lee, J. Lu, P. Noordhuis, M. Smelyanskiy, L. Xiong 和 X. Wang。2018 年。Facebook 的应用机器学习: 数据中心基础设施视角。2018 年 IEEE 高性能计算机架构国际研讨会 (HPCA)。620–629。[20] 何克, 张 X, 任胜, 孙静。2016. 用于图像识别的深度残差学习。2016 年 IEEE 计算机视觉和模式识别会议 (CVPR)。770–778。[21] Kevin Hsieh, Ganesh Ananthanarayanan, Peter Bodik, Shivaram Venkataraman, Paramvir Bahl, Matthai Philipose, Phillip B. Gibbons 和 Onur Mutlu。2018. 焦点: 低延迟、低成本查询大型视频数据集。第 12 届 USENIX 操作系统设计与实现 (OSDI) 会议论文集。USENIX 协会, 269–286。[22] 胡春, 包伟, 王德, 刘锋。2019. 用于边缘推理加速的动态自适应 DNN 手术。在 IEEE INFOCOM 2019 - IEEE 计算机通信会议。1423–1431。[23] 黄高, 陈丹露, 李天红, Felix Wu, Laurens van der Maaten, Kilian Weinberger。2018. 用于资源高效图像分类的多尺度密集网络。在国际学习表征会议 (ICLR) 中。[24] Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv 和 Yoshua Bengio。2017. 量化神经网络: 使用低精度权重和激活训练神经网络。J. 马赫·学习。资源。18, 1 (2017), 6869–6898。[25] 安德烈·伊格纳托夫、拉杜·蒂莫特、安德烈·库里克、杨承洙、王科、菲利克斯·鲍姆、麦克斯·吴、徐立荣和吕克·范·古尔。2019 年。人工智能基准: 2019 年智能手机深度学习的全部内容。在国际计算机视觉会议 (ICCV) 研讨会上。[26] 英国互联网服务提供商。2020. 4G 移动网络体验报告。https://www.opensignal.com/reports/2019/04/uk/mobile-network-experience。[27] 英国互联网服务提供商。2020. 5G 移动网络报告。https://www.opensignal.com/2020/02/20/how-att-sprint-t-mobile-and-verizon-differ-in-their-early-5g-approach。[28] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam 和 D. Kalenichenko。2018. 用于高效整数算术推理的神经网络的量化和训练。在 IEEE 计算机视觉和模式识别会议 (CVPR) 中。2704–2713。[29] 郑赫珍, Hy

- Compiler. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 448–460.
- [31] Jian Ouyang, Shiding Lin, Wei Qi, Yong Wang, Bo Yu, and Song Jiang. 2014. SDA: Software-defined accelerator for large-scale DNN systems. In *2014 IEEE Hot Chips 26 Symposium (HCS)*. 1–23.
 - [32] Norman P. Jouppi et al. 2017. In-Datacenter Performance Analysis of a Tensor Processing Unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 1–12.
 - [33] Daniel Kang, John Emmons, Firas Abuzaïd, Peter Bailis, and Matei Zaharia. 2017. NoScope: Optimizing Neural Network Queries over Video at Scale. *Proc. VLDB Endow.* 10, 11 (2017), 1586–1597.
 - [34] Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. 2017. Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 615–629.
 - [35] Yigitcan Kaya, Sanghyun Hong, and Tudor Dumitras. 2019. Shallow-Deep Networks: Understanding and Mitigating Network Overthinking. In *International Conference on Machine Learning (ICML)*. 3301–3310.
 - [36] Youngsok Kim, Joonsung Kim, Dongju Chae, Daehyun Kim, and Jangwoo Kim. 2019. μ Layer: Low Latency On-Device Inference Using Cooperative Single-Layer Acceleration and Processor-Friendly Quantization. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 45:1–45:15.
 - [37] A. Kouris and C. Bouganis. 2018. Learning to Fly by MySelf: A Self-Supervised CNN-Based Approach for Autonomous Navigation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 1–9.
 - [38] A. Kouris, S. I. Venieris, and C. Bouganis. 2018. CascadeCNN: Pushing the Performance Limits of Quantisation in Convolutional Neural Networks. In *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. 155–1557.
 - [39] A. Kouris, S. I. Venieris, and C. Bouganis. 2020. A Throughput-Latency Co-Optimised Cascade of Convolutional Neural Network Classifiers. In *2020 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1656–1661.
 - [40] C. Kozyrakis. 2013. Resource Efficient Computing for Warehouse-scale Data-centers. In *2013 Design, Automation Test in Europe Conference Exhibition (DATE)*. 1351–1356.
 - [41] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. *Learning multiple layers of features from tiny images*. Technical Report.
 - [42] V. K. Kukkala, J. Tunnell, S. Pasricha, and T. Bradley. 2018. Advanced Driver-Assistance Systems: A Path Toward Autonomous Vehicles. *IEEE Consumer Electronics Magazine* 7, 5 (2018), 18–25.
 - [43] N. D. Lane, S. Bhattacharya, P. Georgiev, C. Forlivesi, L. Jiao, L. Qendro, and F. Kawsar. 2016. DeepX: A Software Accelerator for Low-Power Deep Learning Inference on Mobile Devices. In *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. 1–12.
 - [44] Stefanos Laskaridis, Stylianos I. Venieris, Hyeji Kim, and Nicholas D. Lane. 2020. HAPI: Hardware-Aware Progressive Inference. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.
 - [45] Royson Lee, Stylianos I. Venieris, Lukasz Dudziak, Sourav Bhattacharya, and Nicholas D. Lane. 2019. MobiSR: Efficient On-Device Super-Resolution Through Heterogeneous Mobile Processors. In *The 25th Annual International Conference on Mobile Computing and Networking (MobiCom)*.
 - [46] E. Li, L. Zeng, Z. Zhou, and X. Chen. 2020. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing. *IEEE Transactions on Wireless Communications (TWC)* (2020), 447–457.
 - [47] Hongshan Li, Chenghao Hu, Jingyan Jiang, Zhi Wang, Yonggang Wen, and Wenwu Zhu. 2019. JALAD: Joint Accuracy-And Latency-Aware Deep Structure Decoupling for Edge-Cloud Execution. In *Proceedings of the International Conference on Parallel and Distributed Systems (ICPADS)*. 671–678.
 - [48] Hao Li, Hong Zhang, Xiaojuan Qi, Ruigang Yang, and Gao Huang. 2019. Improved Techniques for Training Adaptive Deep Networks. In *International Conference on Computer Vision (ICCV)*.
 - [49] Yizhi Liu, Yao Wang, Ruofei Yu, Mu Li, Vin Sharma, and Yida Wang. 2019. Optimizing CNN Model Inference on CPUs. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 1025–1040.
 - [50] Jiachen Mao, Xiang Chen, Kent W. Nixon, Christopher Krieger, and Yiran Chen. 2017. MoDNN: Local distributed mobile computing system for Deep Neural Network. *Proceedings of the 2017 Design, Automation and Test in Europe (DATE)* (2017), 1396–1401.
 - [51] Jiachen Mao, Zhongda Yang, Wei Wen, Chunpeng Wu, Linghao Song, Kent W. Nixon, Xiang Chen, Hai Li, and Yiran Chen. 2017. MoDNN: A distributed mobile system with enhanced partition and deployment for large-scale DNNs. *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)* (2017), 751–756.
 - [52] R Timothy Marler and Jasbir S Arora. 2004. Survey of multi-objective optimization methods for engineering. *Structural and multidisciplinary optimization* 26, 6 (2004), 369–395.
 - [53] Szymon Migacz. 2017. 8-bit Inference with TensorRT. In *GPU Technology Conference*.
 - [54] Vinod Nair and Geoffrey E Hinton. 2010. Rectified Linear Units improve Restricted Boltzmann Machines. In *International Conference on Machine Learning (ICML)*. 807–814.
 - [55] Intel Nervana. 2020. Nervana’s Early Exit Inference. https://nervanasystems.github.io/distiller/algo_earlyexit.html. [Retrieved: August 25, 2020].
 - [56] Miloš Nikolić, Mostafa Mahmoud, Andreas Moshovos, Yiren Zhao, and Robert Mullins. 2019. Characterizing Sources of Inefficient Computations in Deep Learning Networks. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 165–176.
 - [57] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau, and Remzi Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 57–70.
 - [58] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems (NeurIPS)*. 8026–8037.
 - [59] Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli, and Jascha Sohl-Dickstein. 2017. On the Expressive Power of Deep Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, Vol. 70. 2847–2854.
 - [60] M. Rhu, M. O’Connor, N. Chatterjee, J. Pool, Y. Kwon, and S. W. Keckler. 2018. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 78–91.
 - [61] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 4510–4520.
 - [62] Hardik Sharma, Jongse Park, Divya Mahajan, Emmanuel Amaro, Joon Kyung Kim, Chenkai Shao, Asit Mishra, and Hadi Esmaeilzadeh. 2016. From High-level Deep Neural Models to FPGAs. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 17:1–17:12.
 - [63] K. Simonyan and A. Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. In *International Conference on Learning Representations (ICLR)*.
 - [64] Ashish Singh and Kakali Chatterjee. 2017. Cloud security issues and challenges: A survey. *Journal of Network and Computer Applications* 79 (2017), 88–115.
 - [65] Muthian Sivathanu, Tapan Chugh, Sanjay S. Singapuram, and Lidong Zhou. 2019. Astra: Exploiting Predictability to Optimize Deep Learning. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 909–923.
 - [66] N. Smolyanskiy, A. Kamenev, J. Smith, and S. Birchfield. 2017. Toward low-flying autonomous MAV trail navigation using deep neural networks for environmental awareness. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 4241–4247.
 - [67] Pierre Stock, Armand Joulin, Răfăi Gribonval, Benjamin Graham, and Hervé Jälgou. 2020. And the Bit Goes Down: Revisiting the Quantization of Neural Networks. In *International Conference on Learning Representations (ICLR)*.
 - [68] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander Alemi. 2017. Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning. In *AAAI Conference on Artificial Intelligence*.
 - [69] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. 2016. Rethinking the Inception Architecture for Computer Vision. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2818–2826.
 - [70] Willy Tarreau et al. 2012. HAProxy-the reliable, high-performance TCP/HTTP load balancer.
 - [71] Ben Taylor, Vicent Sanz Marco, Willy Wolff, Yehia Elkhatib, and Zheng Wang. 2018. Adaptive Deep Learning Model Selection on Embedded Systems. In *Proceedings of the 19th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES) (LCTES 2018)*. 31â$43.
 - [72] Surat Teerapittayanon, Bradley McDanel, and HT Kung. 2016. BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks. In *2016 23rd International Conference on Pattern Recognition (ICPR)*. 2464–2469.
 - [73] S. Teerapittayanon, B. McDanel, and H. T. Kung. 2017. Distributed Deep Neural Networks Over the Cloud, the Edge and End Devices. In *2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS)*. 328–339.
 - [74] Tenstorrent. 2020. Tenstorrent’s Grayskull AI Chip. <https://www.tenstorrent.com/technology/>. [Retrieved: August 25, 2020].
 - [75] S. I. Venieris and C. Bouganis. 2019. fpgaConvNet: Mapping Regular and Irregular Convolutional Neural Networks on FPGAs. *IEEE Transactions on Neural Networks and Learning Systems (TNNLS)* 30, 2 (2019), 326–342.
 - [76] Liang Wang, Mario Almeida, Jeremy Blackburn, and Jon Crowcroft. 2016. C3PO: Computation Congestion Control (PrOactive). In *Proceedings of the 3rd ACM*

编译器。在第二十三届编程语言和操作系统架构支持国际会议 (ASPLOS) 的会议记录中。448–460。[31] 欧阳健, 林世鼎, 魏琪, 王勇, 于波, 宋江。2014. SDA: 大规模 DNN 系统的软件定义加速器。2014 年 IEEE Hot Chips 26 研讨会 (HCS)。1-23。[32] 诺曼·P·乔皮 (Norman P. Jouppi) 等人。2017。张量处理单元的数据中心性能分析。第 44 届计算机体系结构国际研讨会 (ISCA) 论文集。ACM, 1-12。[33] Daniel Kang, John Emmons, Firas Abuzaid, Peter Bailis 和 Matei Zaharia。2017。NoScope: 大规模优化视频上的神经网络查询。过程。VLDB 捐赠。10, 11 (2017), 1586–1597。[34] 康一平、约翰·豪斯瓦尔德、曹高、奥斯汀·罗文斯基、特雷弗·马奇、杰森·马尔斯和唐凌嘉。2017。神经外科医生: 云和移动边缘之间的协作智能。第二十二届编程语言和操作系统架构支持国际会议 (ASPLOS) 论文集。615–629。[35] Yigitcan Kaya, Sanghyun Hong 和 Tudor Dumitras。2019。浅层-深层网络: 理解和减轻网络过度思考。在国际机器学习会议 (ICML) 中。3301–3310。[36] Youngsok Kim, Joonsung Kim, Dongju Chae, Daehyun Kim 和 Jangwoo Kim。2019。μLayer: 使用协作单层加速和处理器友好的量化进行低延迟设备上推理。2019 年第十四届 EuroSys 会议记录。45:1–45:15。[37] A. Kouris 和 C. Bouganis。2018 年。《Learning to Fly by MySelf: 一种基于 CNN 的自我监督自主导航方法》。在 IEEE/RSJ 智能机器人与系统国际会议 (IROS) 中。1-9。[38] A. Kouris, S. I. Venieris 和 C. Bouganis。2018。CascadeCNN: 突破卷积神经网络量化的性能极限。2018 年第 28 届国际现场可编程逻辑与应用会议 (FPL)。155–1557。[39] A. Kouris, S. I. Venieris 和 C. Bouganis。2020。吞吐量-延迟协同优化的卷积神经网络分类器级联。2020 年欧洲设计、自动化测试会议展览 (日期)。1656–1661。[40] C.科兹拉基斯。2013。仓库规模数据中心的资源高效计算。2013 年欧洲设计、自动化测试会议展览 (DATE)。1351–1356。[41] 亚历克斯·克里热夫斯基、杰弗里·辛顿等人。2009。从微小图像中学习多层特征。技术报告。[42] V.K. Kukkala, J. Tunne Il, S. Pasricha 和 T. Bradley。2018。高级驾驶辅助系统: 迈向自动驾驶汽车的道路。IEEE 消费电子杂志 7, 5 (2018), 18–25。[43] N. D. Lane, S. Bhattacharya、P. Georgiev、C. Forlivesi、L. Jiao、L. Qendro 和 F. Kawsar。2016。DeepX: 移动设备上低功耗深度学习推理的软件加速器。2016 年第 15 届 ACM/IEEE 传感器网络信息处理国际会议 (IPSN)。1-12。[44] Stefanos Laskaridis, Stylianos I. Venieris, Hyeji Kim 和 Nicholas D. Lane。2020。HAPI: 硬件感知渐进推理。在 IEEE/ACM 国际计算机辅助设计会议 (ICCAD) 中。[45] Royson Lee, Stylianos I. Venieris, Lukasz Dudziak, Sourav Bhattacharya 和 Nicholas D. Lane。2019。MobiSR: 通过异构移动处理器实现高效的设备上超分辨率。第 25 届年度移动计算和网络国际会议 (MobiCom)。[46] E. Li, L. Zeng, Z. Zhou, X. Chen。2020。边缘人工智能: 通过边缘计算按需加速深度学习推理。IEEE 无线通信汇刊 (TWC) (2020), 447–457。[47] 李红山, 胡成浩, 江敬艳, 王志, 温永刚, 朱文武。2019。JALAD: 边缘云执行的联合精度和延迟感知深度结构解耦。国际并行和分布式系统会议 (ICPADS) 的会议记录。671–678。[48] 李浩, 张宏, 齐小娟, 杨瑞刚, 黄高。2019。训练自适应深度网络的改进技术。在国际计算机视觉会议 (ICCV) 上。[49] 刘益智, 王耀, 于若飞, 李木, Vin Sharma, 王义达。2019。优化 CPU 上的 CNN 模型推理。2019 年 USENIX 年度技术会议 (USENIX ATC 19)。1025–1040。[50] 毛嘉辰, 陈翔, 肯特·W·尼克松, 克里斯托弗·克里格, 陈怡然。2017。MoDNN: 深度神经网络的本地分布式移动计算系统。2017 年欧洲设计、自动化和测试论文集 (日期) (2017 年), 1396–1401。[51] 毛佳辰, 杨忠达, 文伟, 吴春鹏, 宋凌浩, 肯特·尼克松, 陈翔, 李海, 陈怡然。2017。MeDNN: 一种分布式移动系统, 具有针对大规模 DNN 的增强分区和部署。IEEE/ACM 国际计算机辅助设计会议 (ICCAD) (2017), 751–756。[52] R.蒂莫西·马勒和贾斯比·S·阿罗拉。2004。调查

[54] 维诺德·奈尔和杰弗里·E·辛顿。2010。整线性单元改进了受限玻尔兹曼机。在国际机器学习会议 (ICML) 中。807–814。[55] 英特尔 Nervana。2020。Nervana 的提前退出推断。https://nervanasystems.github.io/distiller/algo_earlyexit.html。[检索日期: 2020 年 8 月 25 日]。[56] 米洛什·尼科利奇、莫斯塔法·马哈茂德、安德烈亚斯·莫绍沃斯、赵伊人和罗伯特·穆林斯。2019。描述深度学习网络中无效计算的来源。在 IEEE 国际系统和软件性能分析研讨会 (ISPASS) 中。165–176。[57] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea Arpaci-Dusseau 和 Remzi Arpaci-Dusseau。2018。SOCK: 使用无服务器优化容器快速配置任务。2018 年 USENIX 年度技术会议 (USENIX ATC 18)。57–70。[58] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein、Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai 和苏米特·金塔拉。2019。PyTorch: 命令式高性能深度学习库。神经信息处理系统 (NeurIPS) 的进展。8026–8037。[59] Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli 和 Jascha Sohl-Dickstein。2017。论深度神经网络的表达能力。第 34 届国际机器学习会议 (ICML) 论文集, 卷。70。2847–2854。[60] M. Rhu, M. O’Connor, N. Chatterjee, J. Pool, Y. Kwon 和 S. W. Keckler。2018。压缩 DMA 引擎: 利用激活稀疏性训练深度神经网络。2018 年 IEEE 高性能计算机架构国际研讨会 (HPCA)。78–91。[61] 马克·桑德勒、安德鲁·霍华德、朱梦龙、安德烈·兹莫吉诺夫和陈良杰。2018。MobileNetV2: 倒置残差和线性瓶颈。IEEE 计算机视觉和模式识别会议 (CVPR) 会议记录。4510–4520。[62] Hardik Sharma, Jongse Park, Divya Mahajan, Emmanuel Amaro, Joon Kyung Kim, Chenkai Shao, Asit Mishra 和 Hadi Esmaeilzadeh。2016。从高级深度神经模型到 FPGA。在 IEEE/ACM 国际微架构研讨会 (MICRO) 上。17:1–17:12。[63] K.西蒙扬和 A.齐瑟曼。2015。用于大规模图像识别的超深卷积网络。在国际学习代表会议 (ICLR) 中。[64] 阿什·辛格和卡卡利·查特吉。2017 年。云安全问题和挑战: 一项调查。网络与计算机应用杂志 79 (2017), 88–115。[65] Muthian Sivathanu, Tapan Chugh, Sanjay S. Singapuram 和 Lidong Zhou。2019。Astra: 利用可预测性来优化深度学习。第二十四届编程语言和操作系统架构支持国际会议 (ASPLOS) 论文集。909–923。[66] N. Smolyanskiy, A. Kamenev, J. Smith 和 S. Birchfield。2017。使用深度神经网络实现低空飞行自主 MAV 路径导航以提高环境意识。2017 年 IEEE/RSJ/智能机器人与系统国际会议 (IROS)。4241–4247。[67] 皮埃尔·斯托克、阿曼德·朱林、Rémi Gribonval、本杰明·格雷厄姆和 Hervé Huet。2020 年。比特下降: 重新审视神经网络的量化。在国际学习表征会议 (ICLR) 中。[68] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke 和 Alexander Alemi。2017。Inception-v4, Inception-ResNet 以及残余连接对学习的影响。在 AAAI 人工智能大会上。[69] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens 和 Z. Wojna。2016。重新思考计算机视觉的初始架构。在 IEEE 计算机视觉和模式识别会议 (CVPR) 中。2818–2826。[70] 威利·塔罗等人。2012。HAProxy - 可靠的高性能 TCP/HTTP 负载均衡器。[71] Ben Taylor, Vicent Sanz Marco, Willy Wolff, Yehia Elkhatib 和 Zheng Wang。2018。嵌入式系统上的自适应深度学习模型选择。第 19 届 ACM SIGPLAN/SIGBED 国际嵌入式系统语言、编译器和工具会议 (LCTES) 论文集 (LCTES 2018)。31 至 43。[72] Surat Teerapittayanon, Bradley McDanel 和 HT Kung。2016。BranchyNet: 通过早期退出深度神经网络进行快速推理。2016 年第 23 届国际模式识别会议 (ICPR)。2464–2469。[73] S. Teerapittayanon, B. McDanel 和 H. T. Kung。2017。云、边缘和终端设备上的分布式深度神经网络。2017 年 IEEE 第 37 届分布式计算系统国际会议 (ICDCS)。328–339。[74] 坦托伦特。2020。Tenstorrent 的 Grayskull 人工智能芯片。https://www.tenstorrent.com/技术/。[检索日期: 2020 年 8 月 25 日]。[75] S. I. Venieris 和 C. Bouganis。2019。fpgaConvNet: 映射规则和不规则卷积神经网络

- Conference on Information-Centric Networking (ACM-ICN '16)*. 231–236.
- [77] Liang Wang, Mengyuan Li, Yinqian Zhang, Thomas Ristenpart, and Michael Swift. 2018. Peeking Behind the Curtains of Serverless Platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 133–146.
 - [78] S. Wang, A. Pathania, and T. Mitra. 2020. Neural Network Inference on Mobile SoCs. *IEEE Design Test* (2020).
 - [79] Xuechao Wei, Cody Hao Yu, Peng Zhang, Youxiang Chen, Yuxin Wang, Han Hu, Yun Liang, and Jason Cong. 2017. Automated Systolic Array Architecture Synthesis for High Throughput CNN Inference on FPGAs. In *Proceedings of the 54th Annual Design Automation Conference (DAC)*. 29:1–29:6.
 - [80] C. Wu, D. Brooks, K. Chen, D. Chen, S. Choudhury, M. Dukhan, K. Hazelwood, E. Isaac, Y. Jia, B. Jia, T. Leyvand, H. Lu, Y. Lu, L. Qiao, B. Reagen, J. Spisak, F. Sun, A. Tulloch, P. Vajda, X. Wang, Y. Wang, B. Wasti, Y. Wu, R. Xian, S. Yoo, and P. Zhang. 2019. Machine Learning at Facebook: Understanding Inference at the Edge. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 331–344.
 - [81] Ji Xin, Raphael Tang, Jaejun Lee, Yaoliang Yu, and Jimmy Lin. 2020. DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. Association for Computational Linguistics, 2246–2251.
 - [82] Bing Xu, Naiyan Wang, Tianqi Chen, and Mu Li. 2015. Empirical Evaluation of Rectified Activations in Convolutional Network. In *CoRR*.
 - [83] Linfeng Zhang, Jiebo Song, Anni Gao, Jingwei Chen, Chenglong Bao, and Kaisheng Ma. 2019. Be Your Own Teacher: Improve the Performance of Convolutional Neural Networks via Self Distillation. In *IEEE International Conference on Computer Vision (ICCV)*.
 - [84] Linfeng Zhang, Zhanhong Tan, Jiebo Song, Jingwei Chen, Chenglong Bao, and Kaisheng Ma. 2019. SCAN: A Scalable Neural Networks Framework Towards Compact and Efficient Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [85] Zhuoran Zhao, Kamyar Mirzazad Barijough, and Andreas Gerstlauer. 2018. DeepThings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 37 (2018), 2348–2359.
 - [86] Aojun Zhou, Anbang Yao, Yiwen Guo, Lin Xu, and Yurong Chen. 2017. Incremental Network Quantization: Towards Lossless CNNs with Low-Precision Weights. In *International Conference on Learning Representations (ICLR)*.

以信息为中心的网络会议 (ACM-ICN '16)。231–236。[77] 王亮, 李梦媛, 张银前, 托马斯·里斯滕帕特, 迈克尔·斯威夫特。2018年。一窥无服务器平台的幕后。2018年USENIX年度技术会议 (USENIX ATC 18)。133–146。[78] S. Wang, A. Pathania 和 T. Mitra。2020。移动 SoC 上的神经网络推理。IEEE 设计测试 (2020)。[79] 魏学超, 余浩, 张鹏, 陈友翔, 王雨欣, 胡涵, 梁云, 丛杰森。2017。用于 FPGA 上高吞吐量 CNN 推理的自动脉动阵列架构综合。第 54 届年度设计自动化会议 (DAC) 论文集。29:1–29:6。[80] C. Wu, D. Brooks, K. Chen, D. Chen, S. Choudhury, M. Dukhan, K. Hazelwood, E. Isaac, Y. Jia, B. Jia, T. Leyvand, H. Lu, Y. Lu, L. Qiao, B. Reagen, J. Spisak, F. Sun, A. Tulloch, P. Vajda, X. Wang, Y. Wang, B. Wasti, Y. 吴, R. 西安, S. Yoo 和 P. 张。2019。Facebook 的机器学习: 理解边缘推理。2019年IEEE高性能计算机架构国际研讨会 (HPCA)。331–344。[81] 季欣, Raphael Tang, Jaejun Lee, Yaoliang Yu, Jimmy Lin。2020。DeeBERT: 动态提前退出加速 BERT 推理。计算语言学协会 (ACL) 第 58 届年会记录。协会

计算语言学, 2246–2251。[82] 徐冰, 王乃彦, 陈天琪, 李木。2015。卷积网络中修正激活的实证评估。在 CoRR 中。[83] 张林峰, 宋杰波, 高安妮, 陈静伟, 包成龙, 马凯胜。2019。做你自己的老师: 通过自蒸馏提高卷积神经网络的性能。在 IEEE 国际计算机视觉会议 (ICCV) 上。[84] 张林峰, 谭展宏, 宋杰波, 陈经伟, 鲍成龙, 马凯胜。2019。SCAN: 面向紧凑高效模型的可扩展神经网络框架。神经信息处理系统 (NeurIPS) 的进展。[85] 赵卓然, Kamyar Mirzazad Barijough, Andreas Gerstlauer。2018。DeepThings: 资源受限的物联网边缘集群上的分布式自适应深度学习推理。IEEE 集成电路与系统计算机辅助设计 (TCAD) 37 (2018), 2348–2359。[86] 周傲军, 姚安邦, 郭一文, 徐林, 陈玉蓉。2017。增量网络量化: 走向具有低精度权重的无损 CNN。在国际学习表征会议 (ICLR) 中。